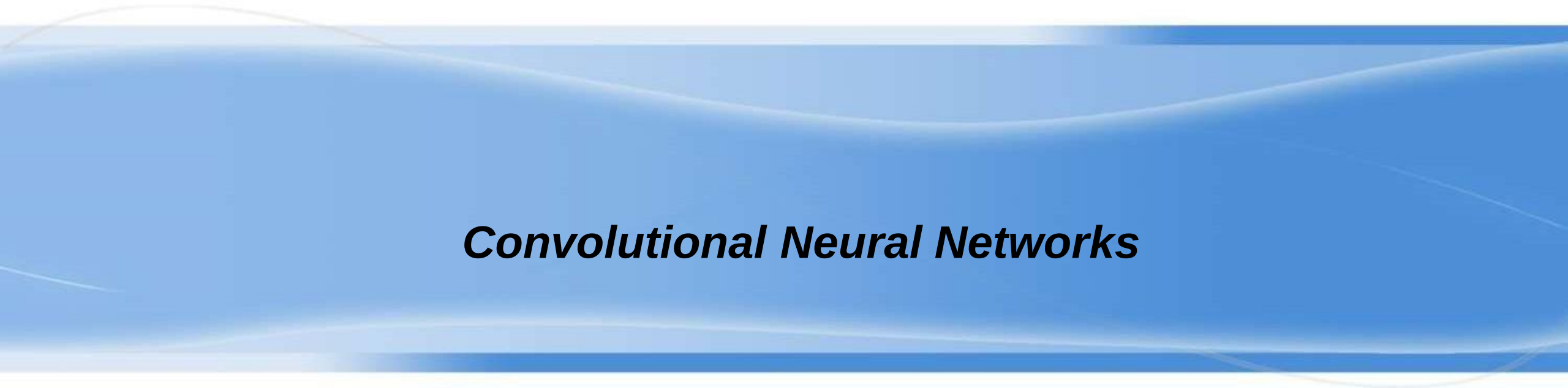
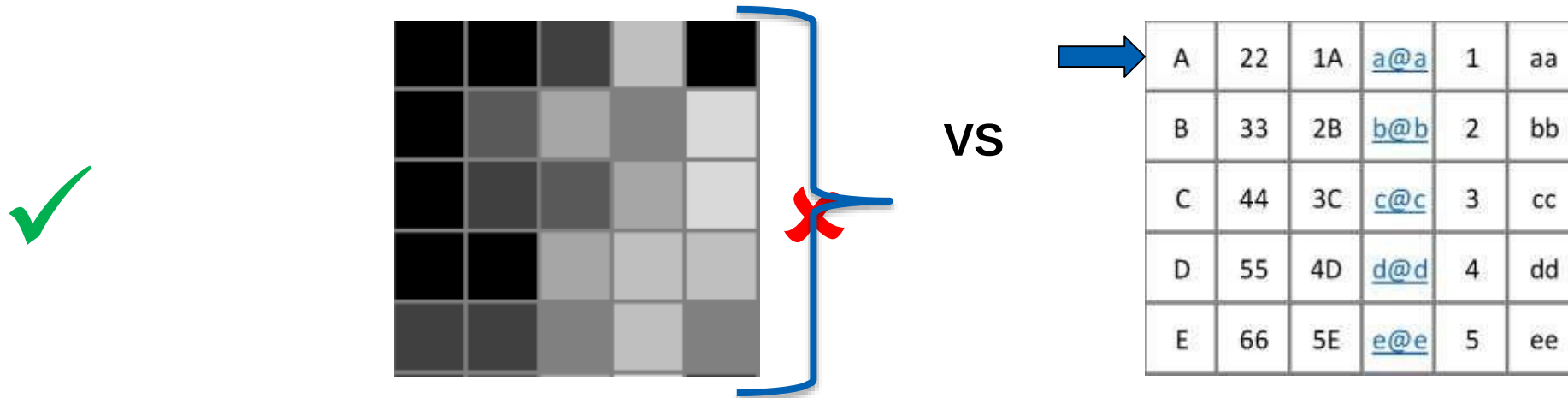


A decorative graphic consisting of a blue band with wavy, fluid lines, resembling water or smoke, set against a white background.

Convolutional Neural Networks

Introduction

- A convolutional neural network (or ConvNet) is a type of feed-forward artificial neural network
- The architecture of a ConvNet is designed to take advantage of the 2D structure of an input image.

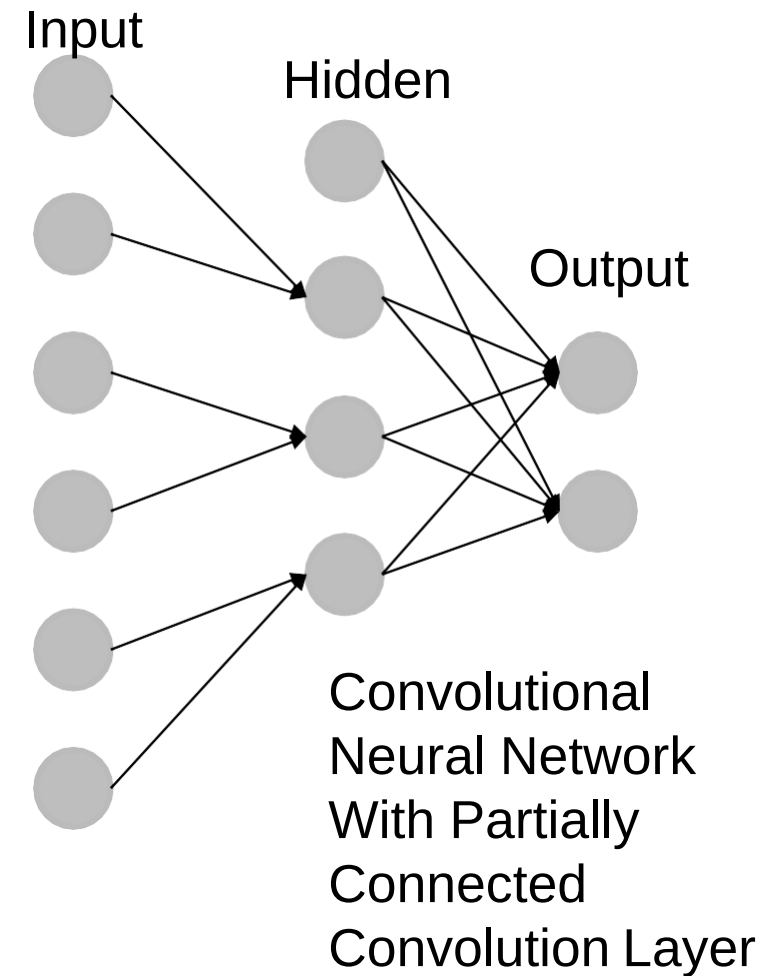
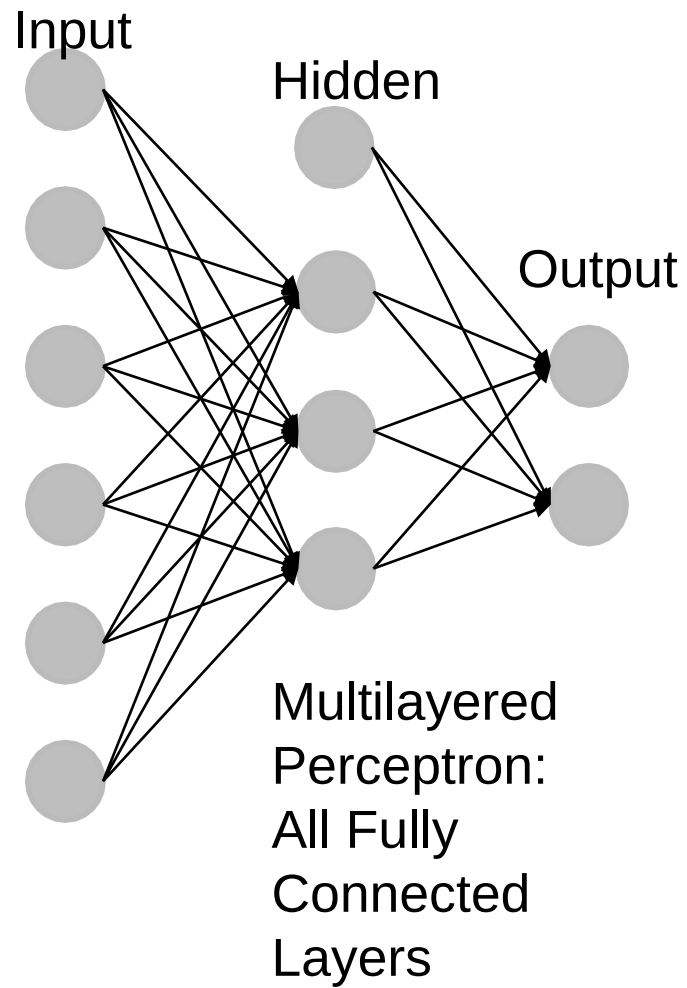


- A ConvNet is comprised of one or more convolutional layers (often with a pooling step) and then followed by one or more fully connected layers as in a standard multilayer neural network.

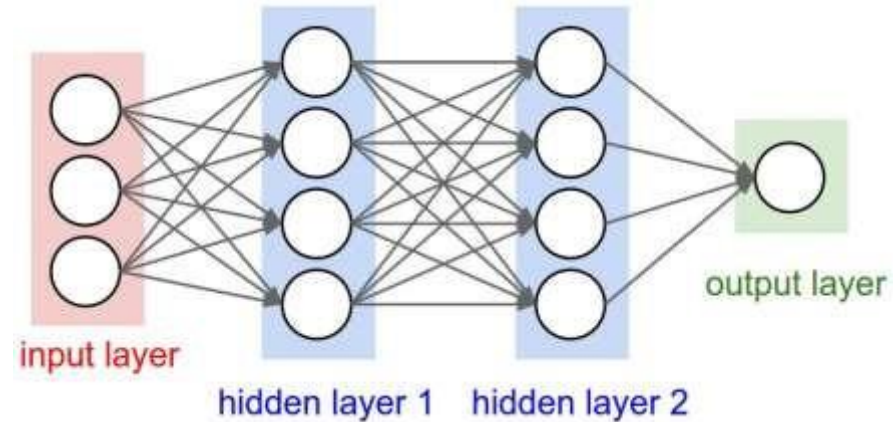
Motivation behind ConvNets

- Consider an image of size $200 \times 200 \times 3$ (200 wide, 200 high, 3 color channels)
 - a **single fully-connected neuron** in a first hidden layer of a regular Neural Network would have $200 \times 200 \times 3 = 120,000$ weights.
 - Due to the presence of several such neurons, this full connectivity is waste and the huge number of parameters would quickly lead to overfitting
- However, in a ConvNet, the neurons in a layer will only be connected to a small region of the layer before it, instead of all of the neurons in a fully-connected manner.
 - the final output layer would have dimensions $1 \times 1 \times N$, because by the end of the ConvNet architecture we will reduce the full image into a single vector of class scores (for N classes), arranged along the depth dimension
- Vanishing Gradient Problem in MLP

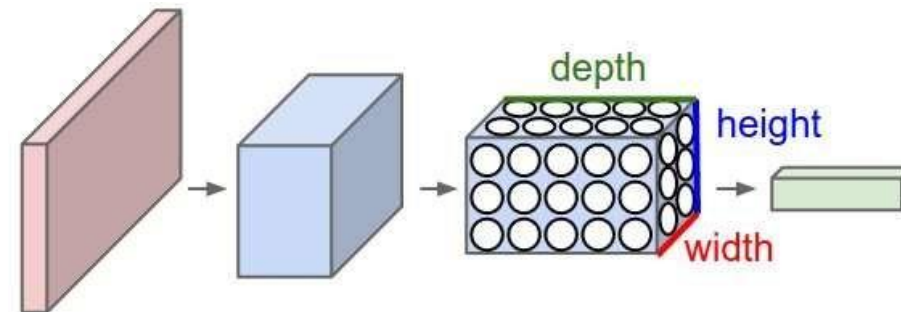
MLP VS ConvNet



- A regular 3-layer Neural Network.



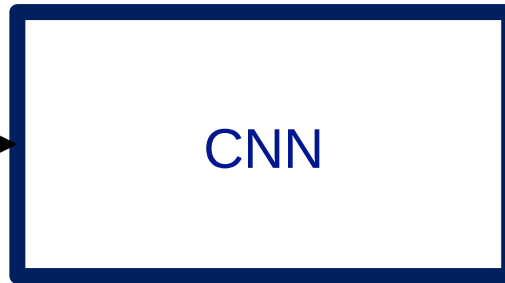
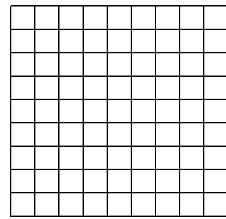
- A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers.



How ConvNet Works

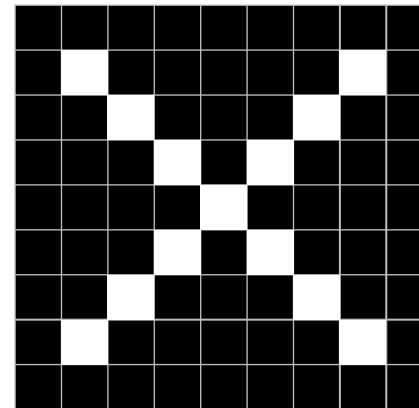
- For example, a ConvNet takes the input as an image which can be classified as 'X' or 'O'

A two-dimensional
array of pixels



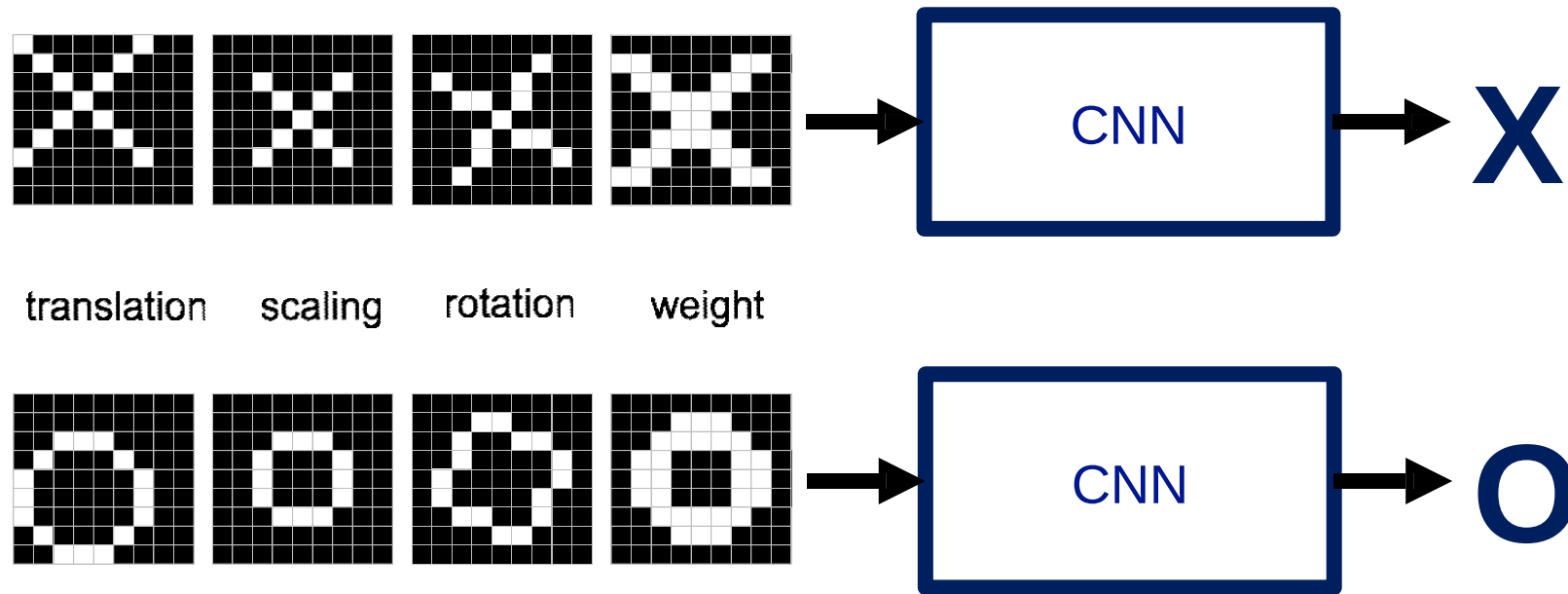
X or **O**

- In a simple case, 'X' would look like:



How ConvNet Works

- What about trickier cases?



How ConvNet Works – What Computer Sees

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

How ConvNet Works

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

How ConvNet Works – What Computer Sees

- Since the pattern doesnot match exactly, the computer will not be able to classify this as ‘X’

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	X	-1	-1	-1	-1	X	X	-1
-1	X	X	-1	-1	X	X	-1	-1
-1	-1	X	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	X	-1	-1
-1	-1	X	X	-1	-1	X	X	-1
-1	X	X	-1	-1	-1	-1	X	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

ConvNet Layers (At a Glance)

- CONV layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume.
- RELU layer will apply an elementwise activation function, such as the $\max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged.
- POOL layer will perform a downsampling operation along the spatial dimensions (width, height).
- FC (i.e. fully-connected) layer will compute the class scores, resulting in volume of size $[1 \times 1 \times N]$, where each of the N numbers correspond to a class score, such as among the N categories.

Recall – What Computer Sees

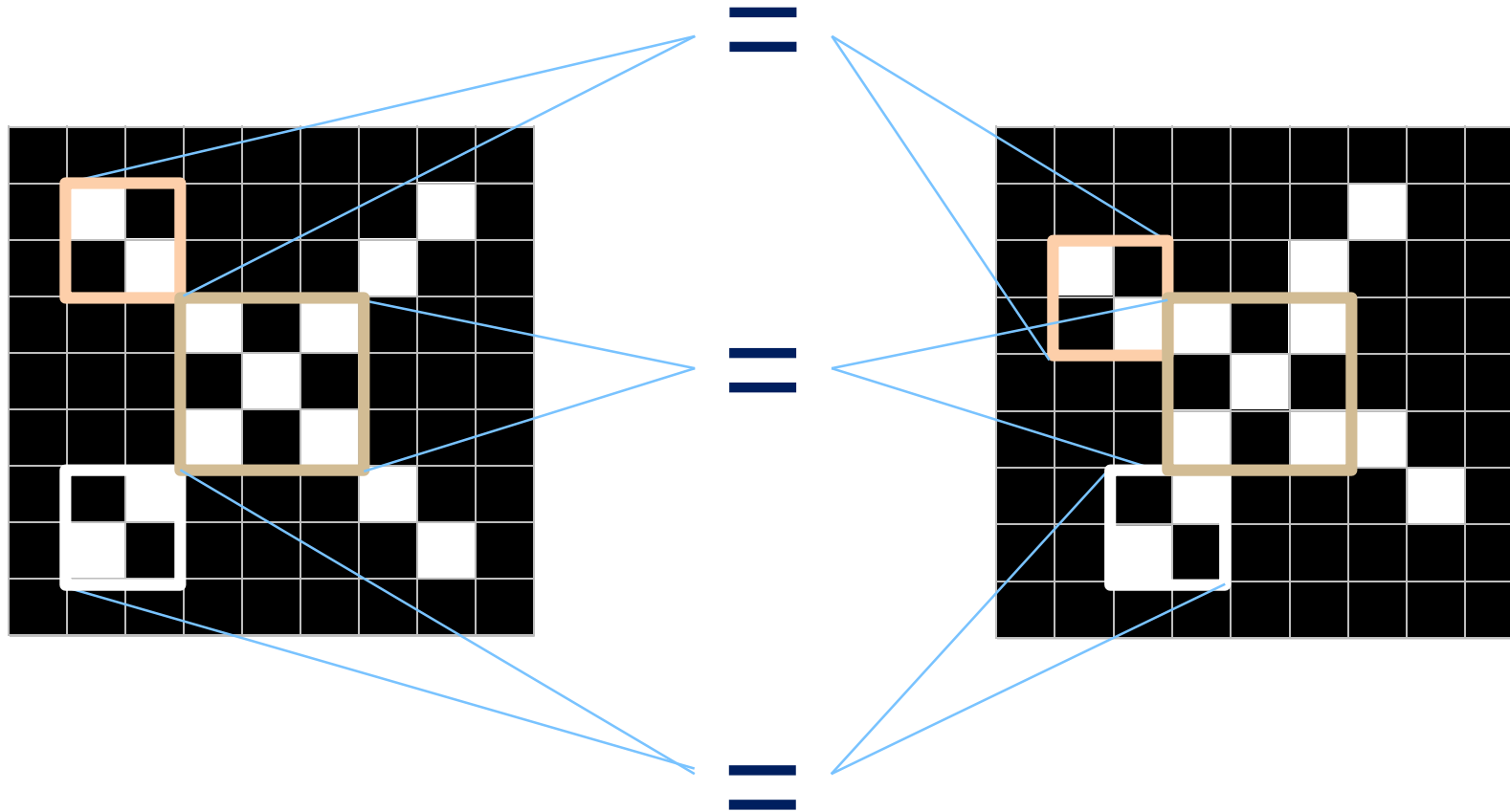
- Since the pattern doesnot match exactly, the computer will not be able to classify this as 'X'

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	X	-1	-1	-1	-1	X	X	-1
-1	X	X	-1	-1	X	X	-1	-1
-1	-1	X	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	X	-1	-1
-1	-1	X	X	-1	-1	X	X	-1
-1	X	X	-1	-1	-1	-1	X	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

- What got changed?

Convolutional Layer

- Convolution layer will work to identify patterns (features) instead of individual pixels



Convolutional Layer - Filters

- The CONV layer's parameters consist of a set of learnable filters.
- Every filter is small spatially (along width and height), but extends through the full depth of the input volume.
- During the forward pass, we slide (more precisely, convolve) each filter across the width and height of the input volume and compute dot products between the entries of the filter and the input at any position.

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1

Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5			

4x4

Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4x4

python: conv-forward
tensorflow: tf.nn.conv2d
keras: Conv2D

Learning to detect edges

1	0	-1
1	0	-1
1	0	-1



3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

→

1	0	-1
2	0	-2
1	0	-1

Sobel filter



convolution
*

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

3×3

=

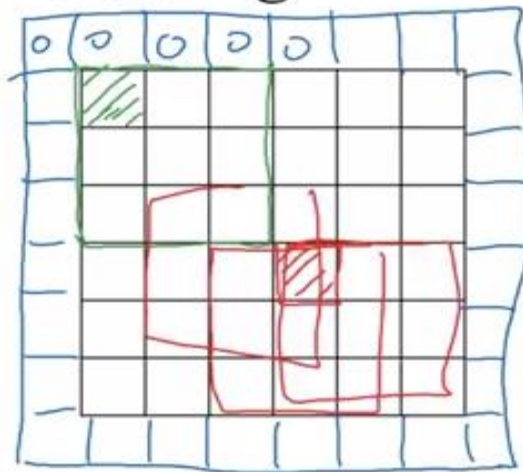
45°
70°
77°

3	0	-3
10	0	-10
3	0	-3

Scharr filter

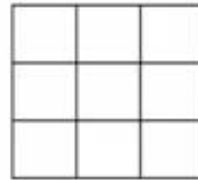


Padding



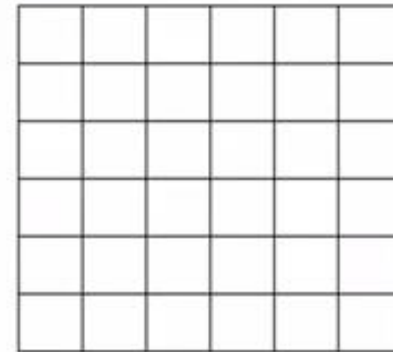
- shrinky output
- throw away info from edge

*



3x3
f x f

=



6x6

$$\frac{6 \times 6}{n \times n} \rightarrow 8 \times 8$$

$$p = \text{padding}$$

$$n - f + 1 \times n - f + 1$$

$$6 - 3 + 1 = 4$$

$$\rightarrow \underline{\underline{4 \times 4}}$$

$$n + 2p - f + 1 \times n + 2p - f + 1$$

$$6 + 2 - 3 + 1 \times \text{---} = 6 \times 6$$

Valid and Same convolutions

→ no padding

“Valid”: $n \times n \quad * \quad f \times f \quad \rightarrow \quad \underline{n - f + 1} \times n - f + 1$
 $6 \times 6 \quad * \quad 3 \times 3 \quad \rightarrow \quad 4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$n + 2p - f + 1 \times n + 2p - f + 1$$

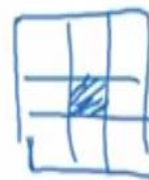
$$n + 2p - f + 1 = n \Rightarrow \boxed{p = \frac{f-1}{2}}$$

$$3 \times 3 \quad p = \frac{3-1}{2} = 1$$

$$\begin{matrix} 5 \times 5 \\ f=5 \end{matrix}$$

$$p=2$$

f is usually odd



1x1
3x3
5x5
7x7

Strided convolution

2	3	7 ³	4 ⁴	6 ⁴	2	9
6	6	9 ¹	8 ⁰	7 ²	4	3
3	4	8 ⁻¹	3 ⁰	8 ³	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

3	4	4
1	0	2
-1	0	3

3x3

stride = 2

=

91		

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3 ³	4 ⁴	8 ⁴	3	8	9	7
7 ¹	8 ⁰	3 ²	6	6	3	4
4 ⁻¹	2 ⁰	1 ³	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

3	4	4
1	0	2
-1	0	3

3x3

stride = 2

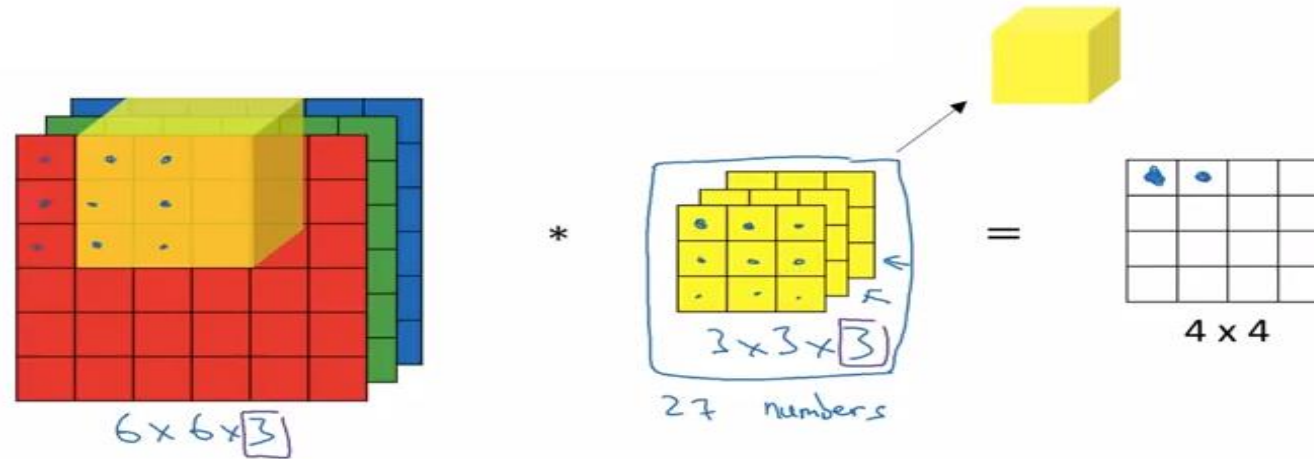
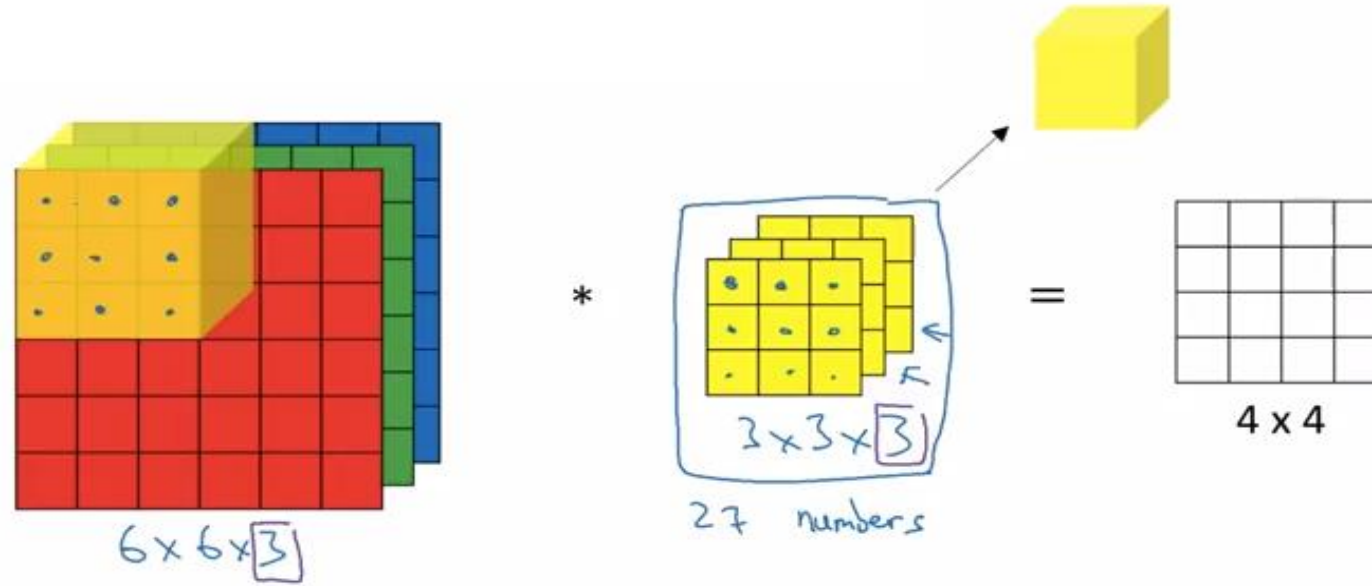
=

91	100	83

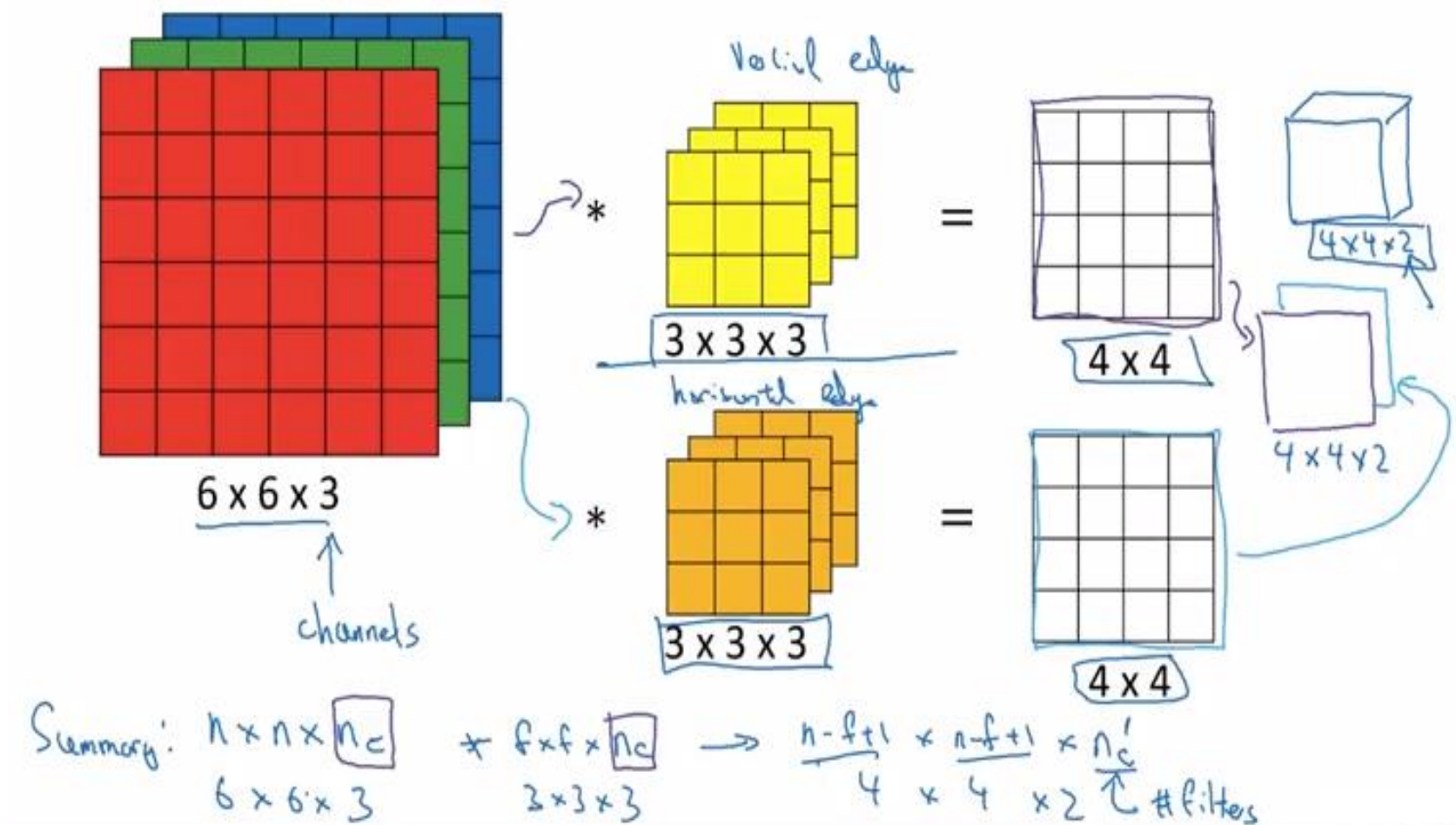
$$\begin{array}{lcl}
 n \times n & * & f \times f \\
 \text{padding } p & & \text{stride } s \\
 & & s = 2
 \end{array}$$

$$\begin{aligned}
 & \frac{n+2p-f}{s} + 1 \quad \times \quad \frac{n+2p-f}{s} + 1 \\
 & \frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3
 \end{aligned}$$

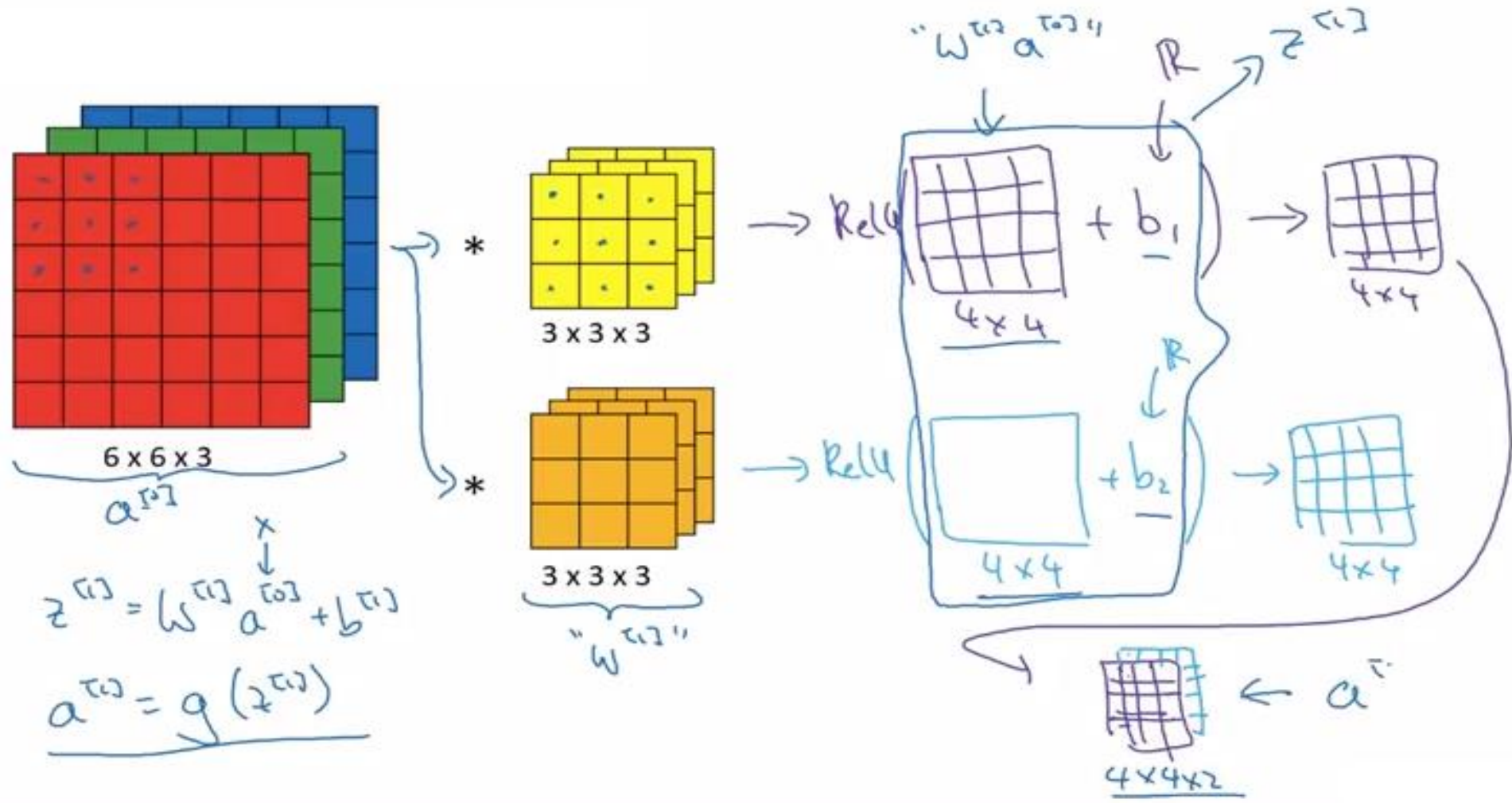
Convolutions on RGB image



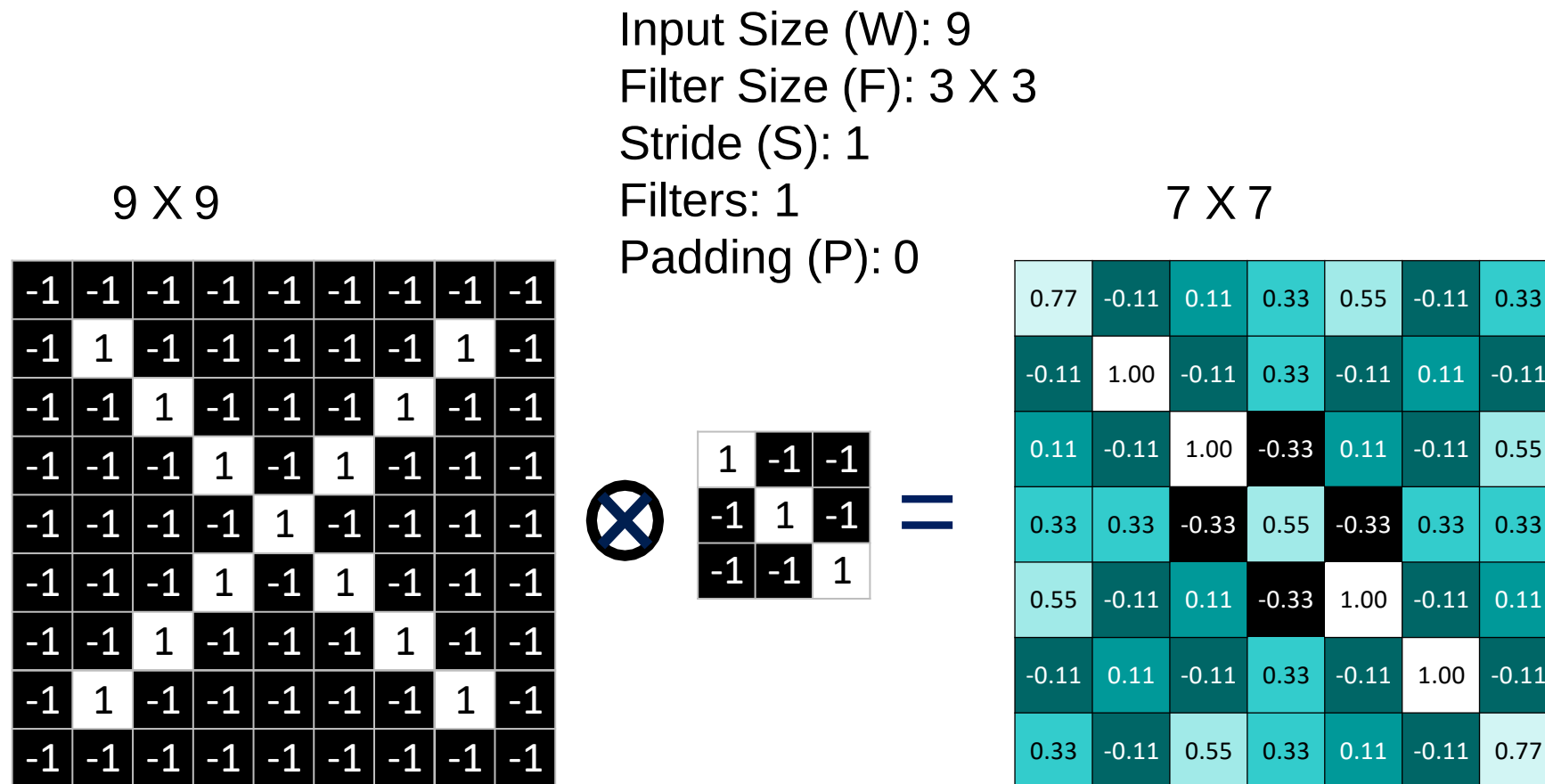
Multiple Filters



Example of a layer



Convolutional Layer – Filters – Computation Example



$$\begin{aligned}\text{Feature Map Size} &= 1 + (W - F + 2P)/S \\ &= 1 + (9 - 3 + 2 \times 0)/1 = 7\end{aligned}$$

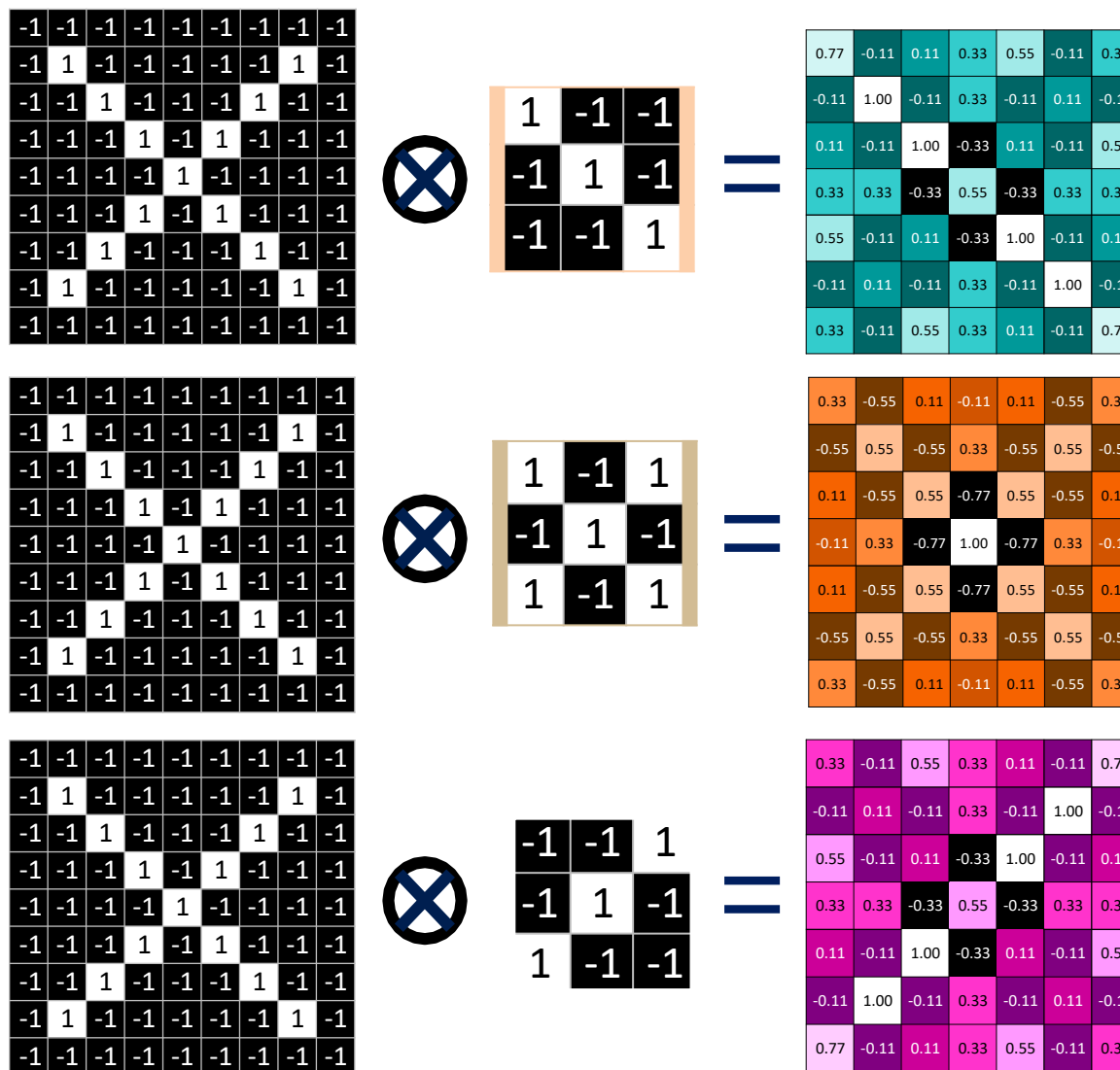
Convolutional Layer – Filters – Output Feature Map

■ Output Feature Map of One complete convolution:

- Filters: 3
- Filter Size: 3 X 3
- Stride: 1

■ Conclusion:

- Input Image:
9 X 9
- Output of Convolution:
7 X 7 X 3



Convolutional Layer – Output

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



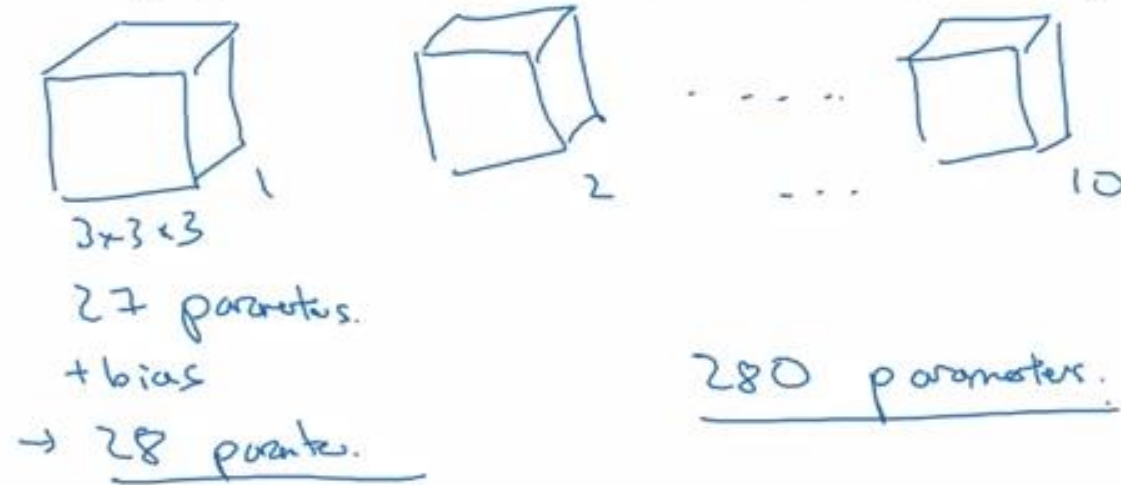
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33

Number of parameters in one layer

If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?



Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

→ Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

Activations: $A^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

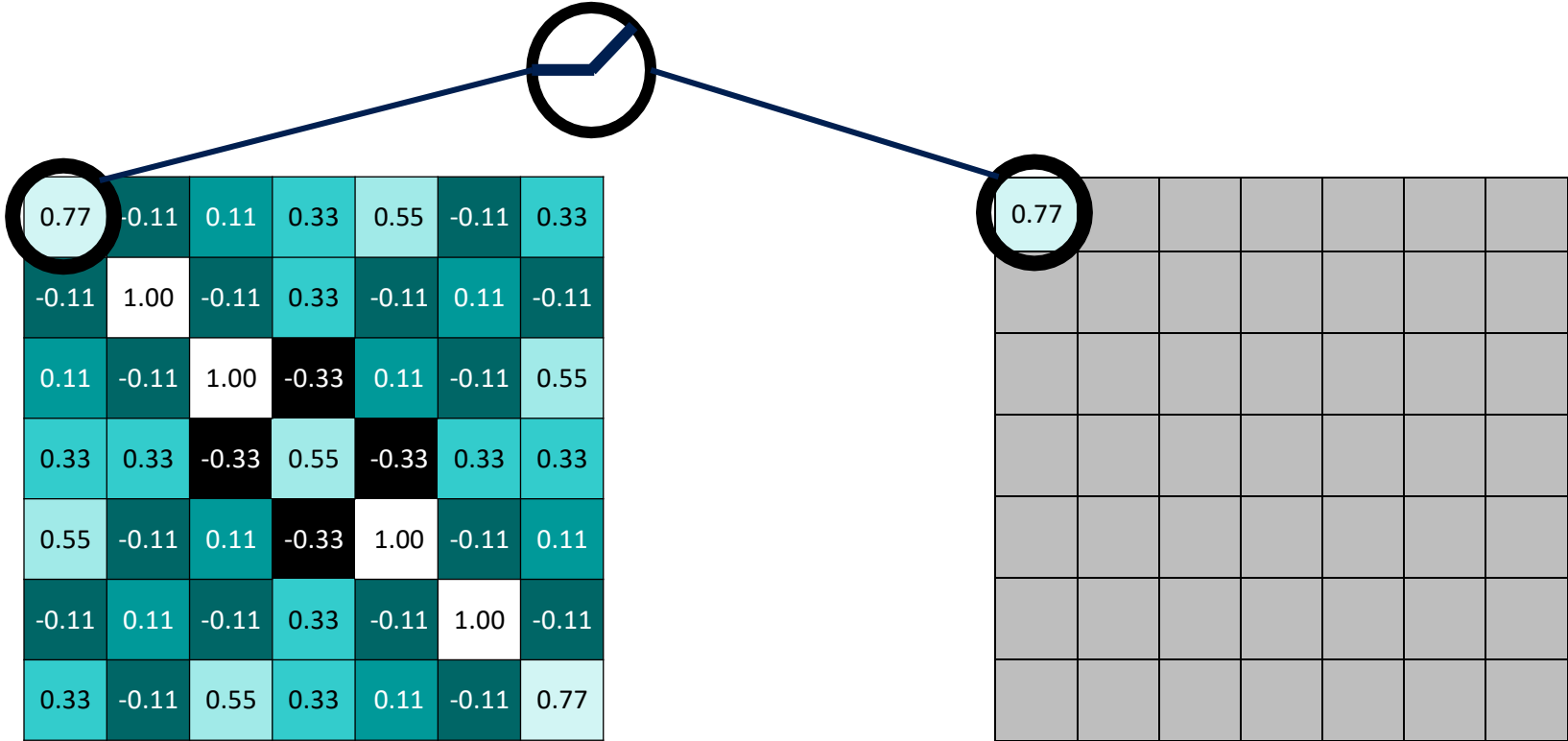
bias: $n_c^{[l]} = (1, 1, 1, \dots, n_c^{[l]})$ ← #filters in layer l.

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$ ←
Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$ ←

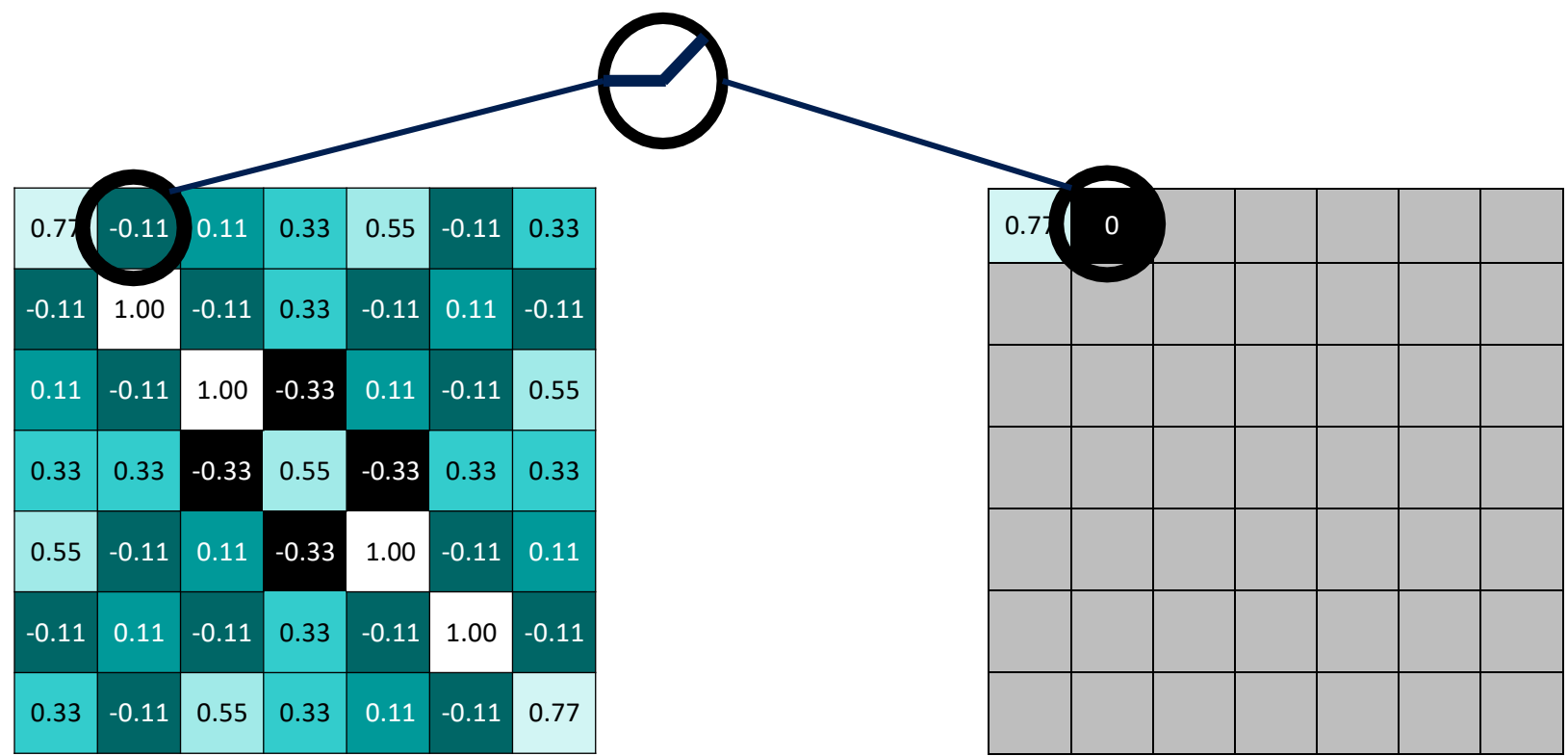
$$n_{HW}^{[l]} = \left\lfloor \frac{n_{HW}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$A^{[l]} \rightarrow m \times \underbrace{n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}}_{n_c \times n_H \times n_W}$$

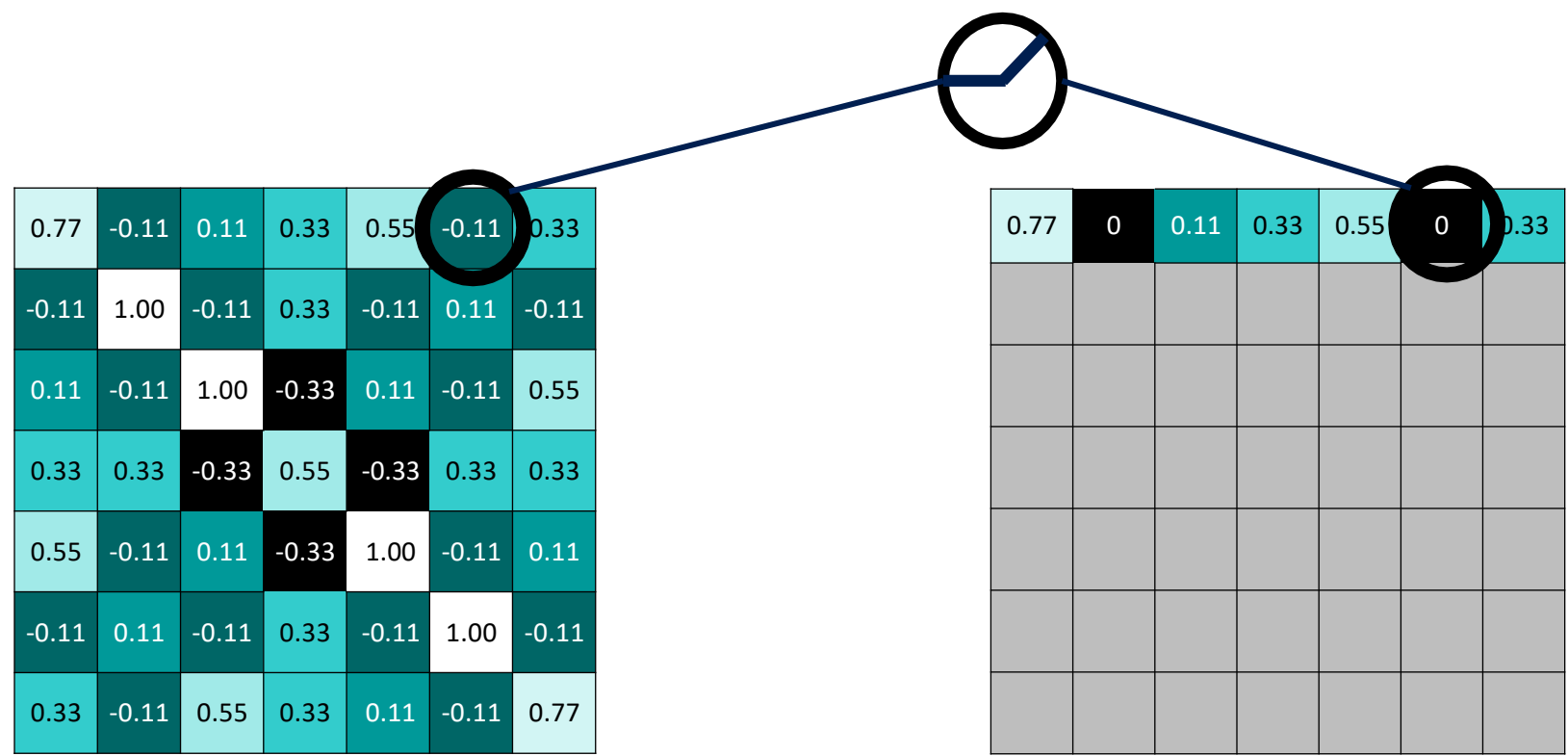
Rectified Linear Units (ReLUs)



Rectified Linear Units (ReLUs)



Rectified Linear Units (ReLUs)

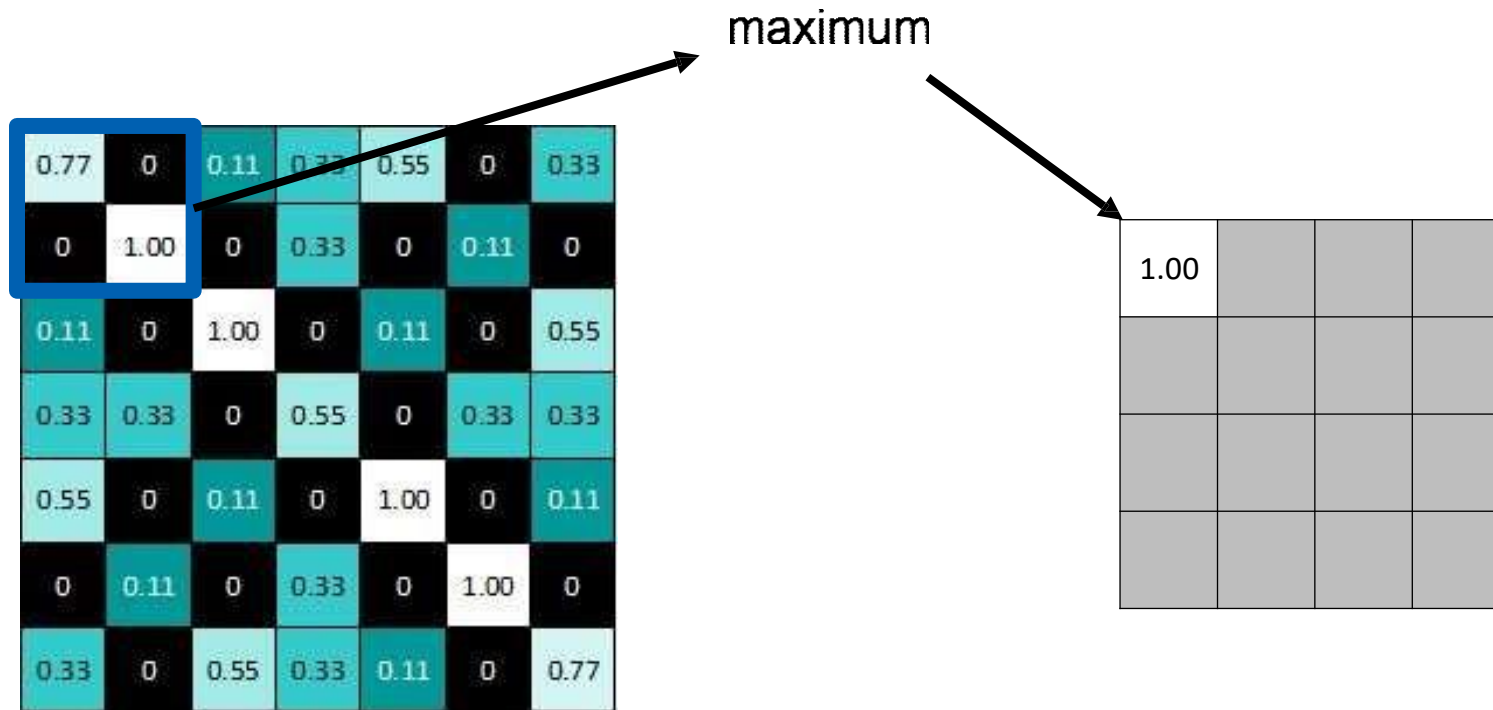


Pooling Layer

- The pooling layers down-sample the previous layers feature map.
- Its function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network
- The pooling layer often uses the Max operation to perform the downsampling process

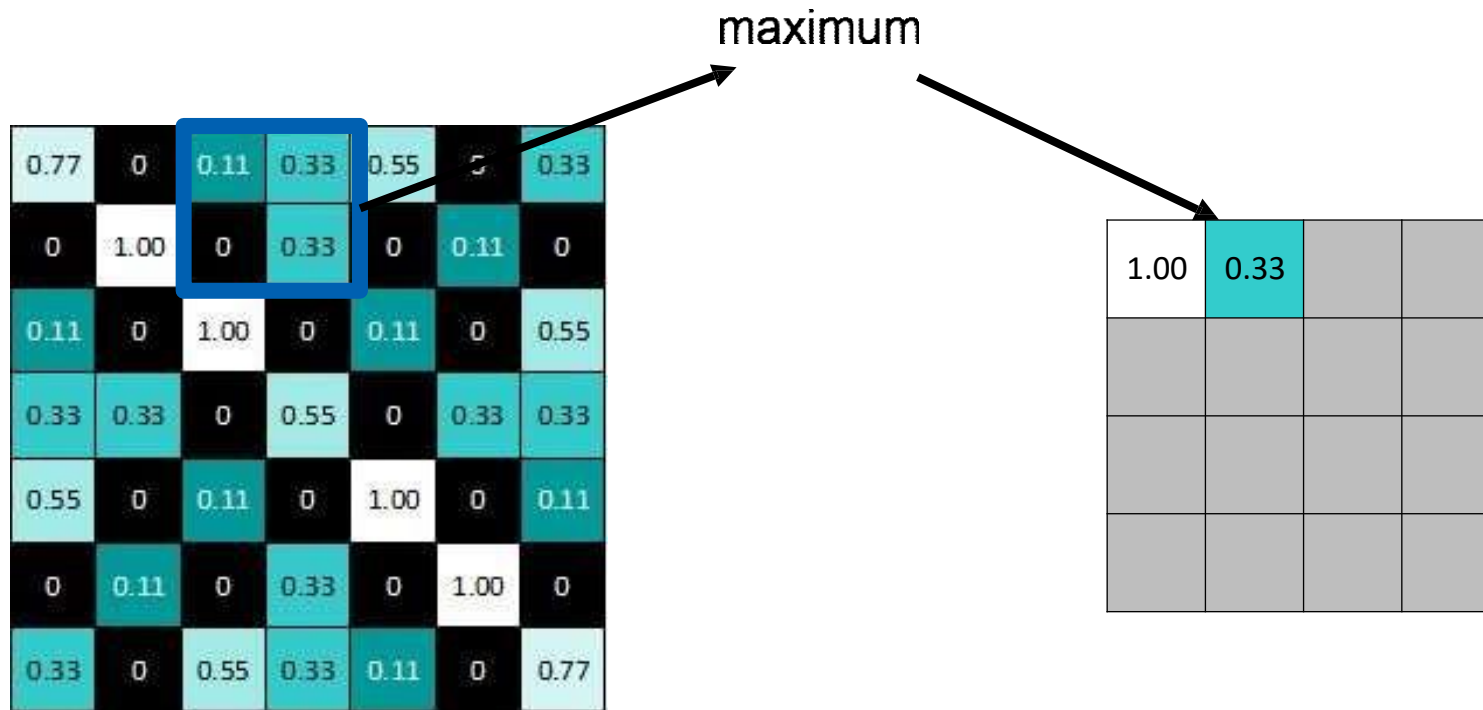
Pooling

- Pooling Filter Size = 2 X 2, Stride = 2



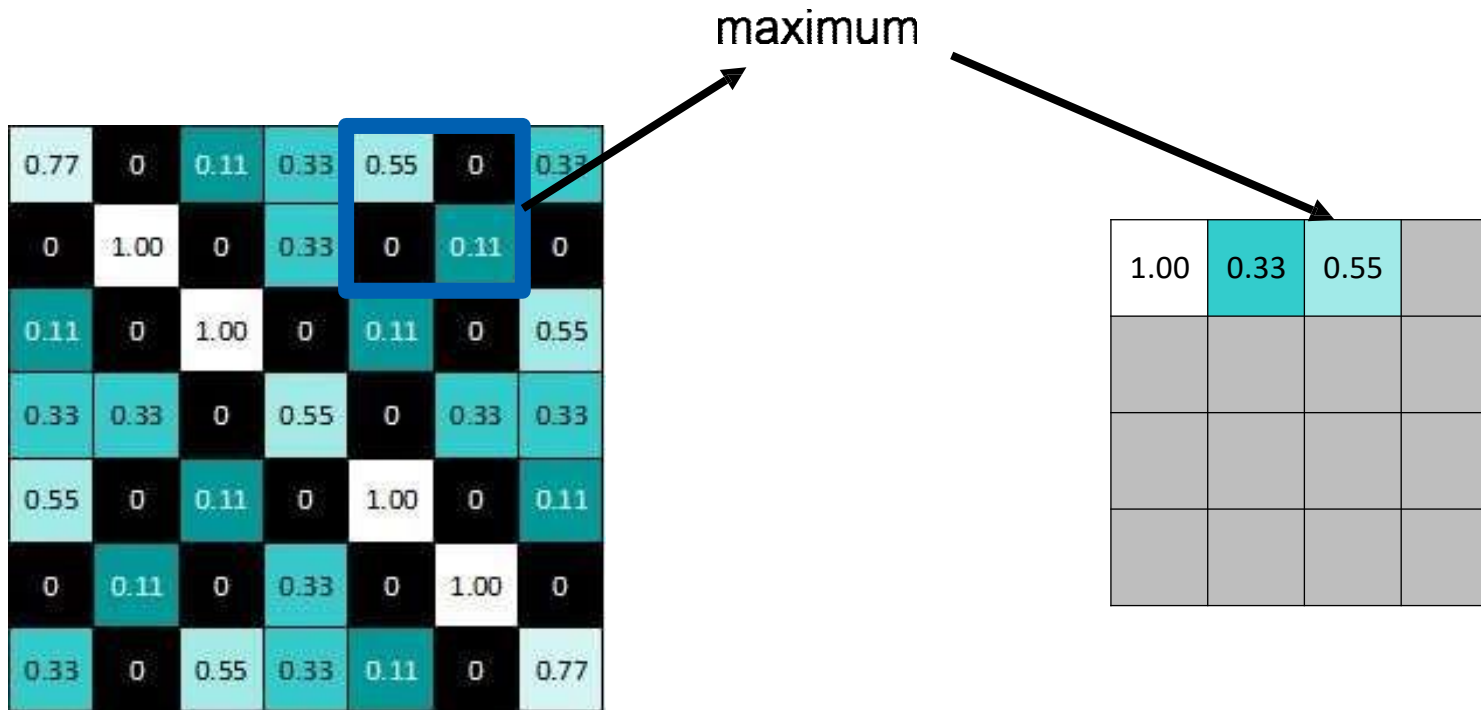
Pooling

- Pooling Filter Size = 2 X 2, Stride = 2



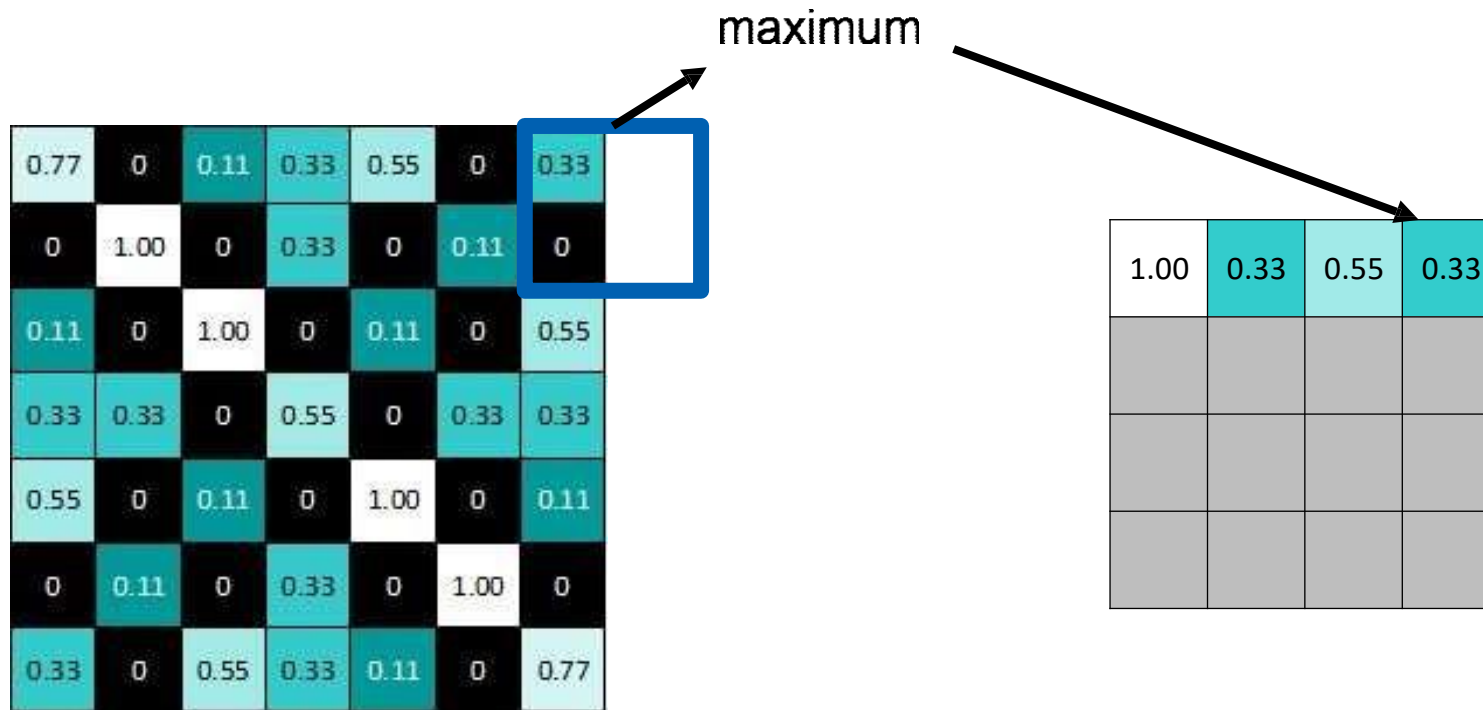
Pooling

- Pooling Filter Size = 2 X 2, Stride = 2



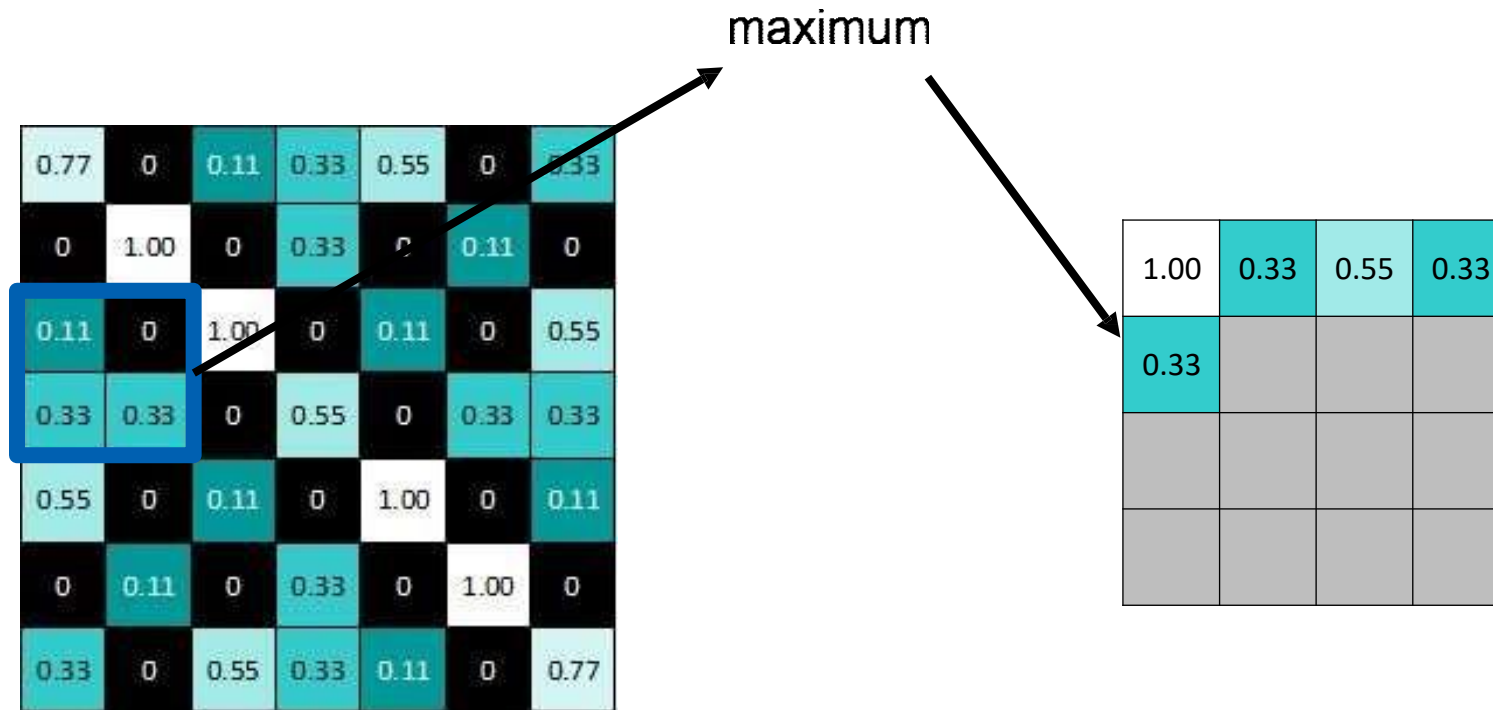
Pooling

- Pooling Filter Size = 2 X 2, Stride = 2



Pooling

- Pooling Filter Size = 2 X 2, Stride = 2



Pooling

- Pooling Filter Size = 2 X 2, Stride = 2

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

max pooling

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

Pooling Layer : Average Pooling

1	3	2	1
2	9	1	1
1	4	2	3
5	6	1	2

→

3.75	1.25
4	2

$$f=2$$
$$s=2$$

Hyperparameters:

f : filter size
s : stride
Max or average pooling

$$f=2, s=2$$
$$f=3, s=2$$

No parameters to learn!

$$n_H \times n_W \times n_C$$

↓

$$\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \times n_C$$

Pooling

A stack of 3 larger images becomes a stack of smaller images.

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.33	0	0.11	0	0.11	0	0.33
0	0.33	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.11	0	1.00	0	0.11	0
0.11	0	0.55	0	0.55	0	0.11
0	0.33	0	0.33	0	0.55	0
0.33	0	0.11	0	0.11	0	0.33



0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

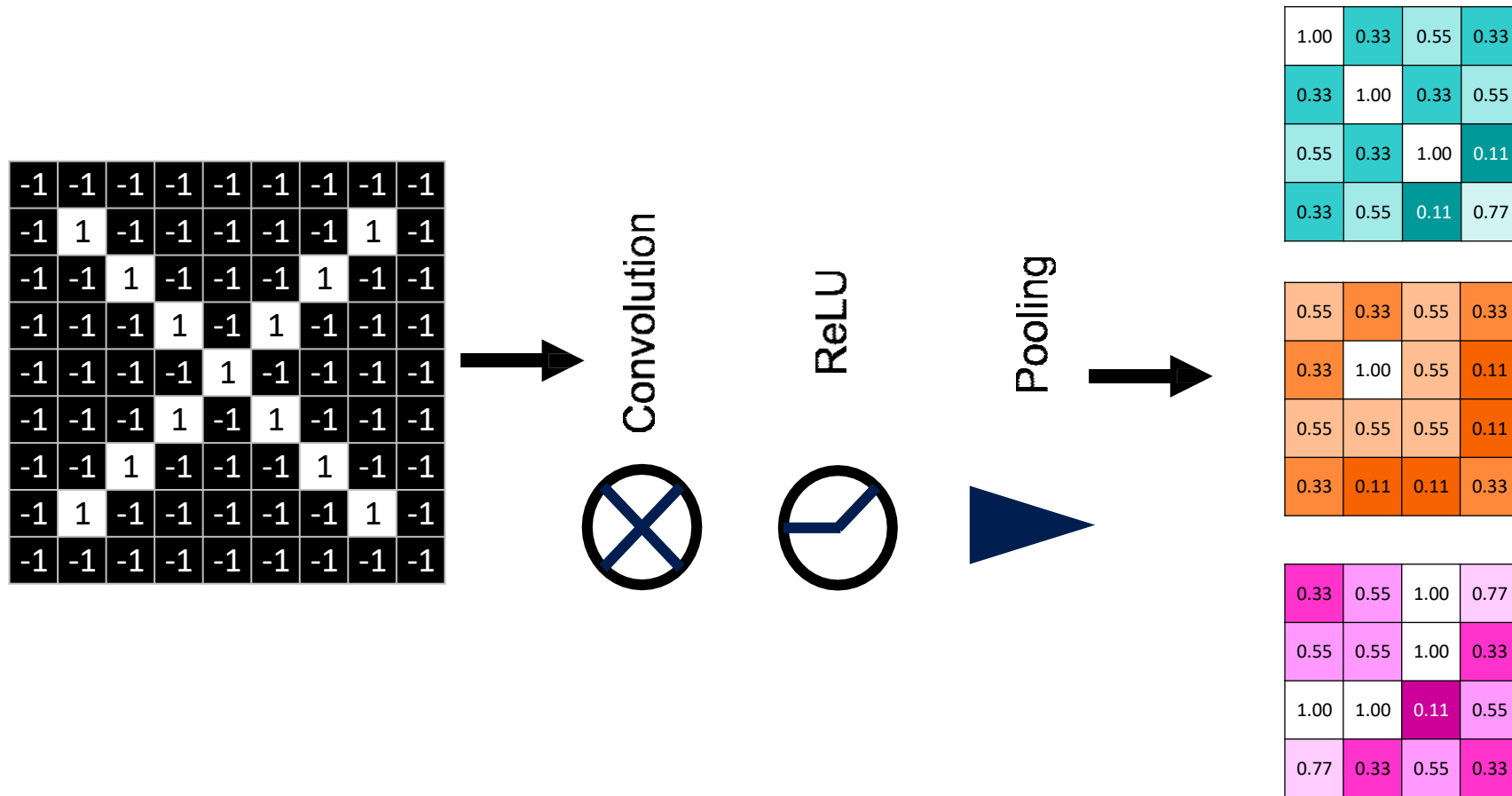
0.33	0	0.55	0.33	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.55	0	0.11	0	1.00	0	0.11
0.33	0.33	0	0.55	0	0.33	0.33
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.11	0
0.77	0	0.11	0.33	0.55	0	0.33



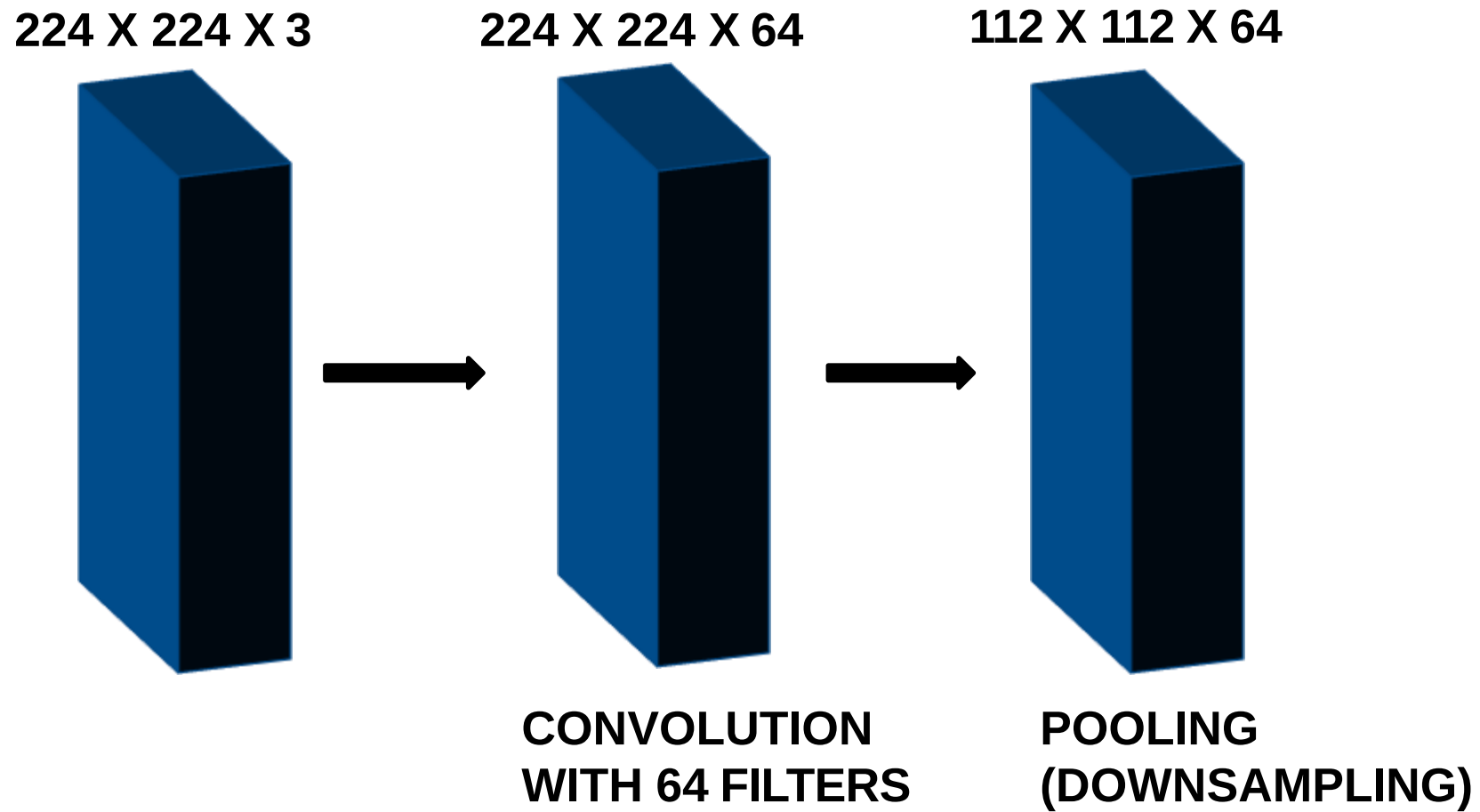
0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

Layers get stacked

The output of one becomes the input of the next.

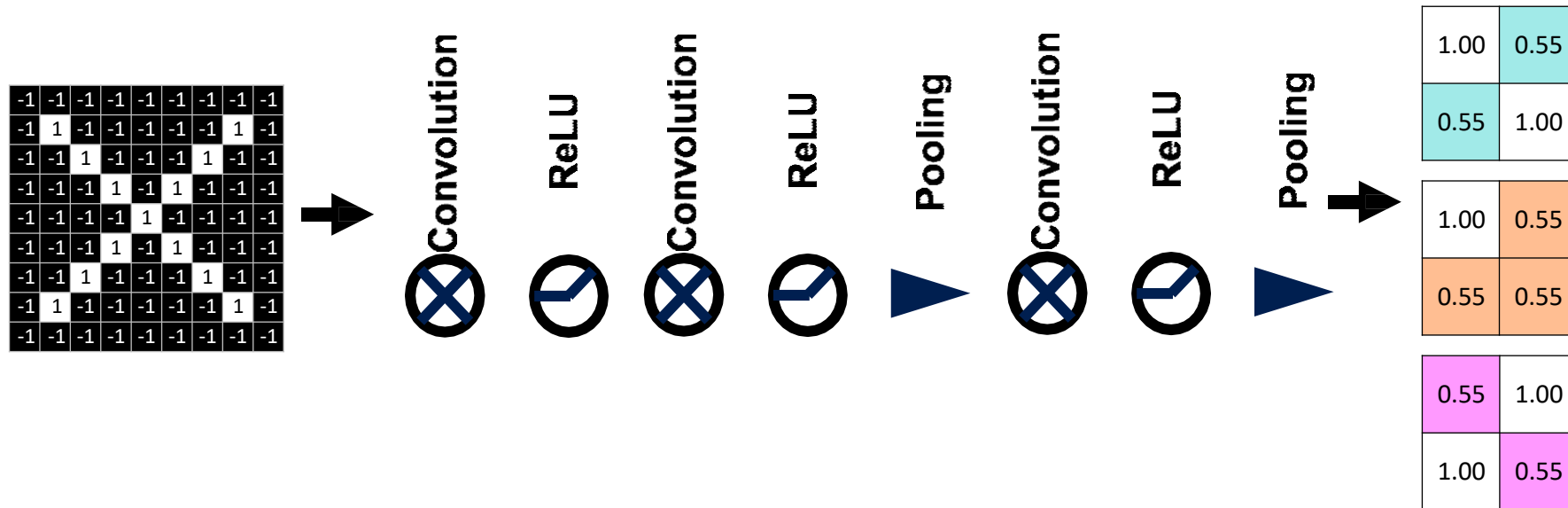


Layers Get Stacked - Example



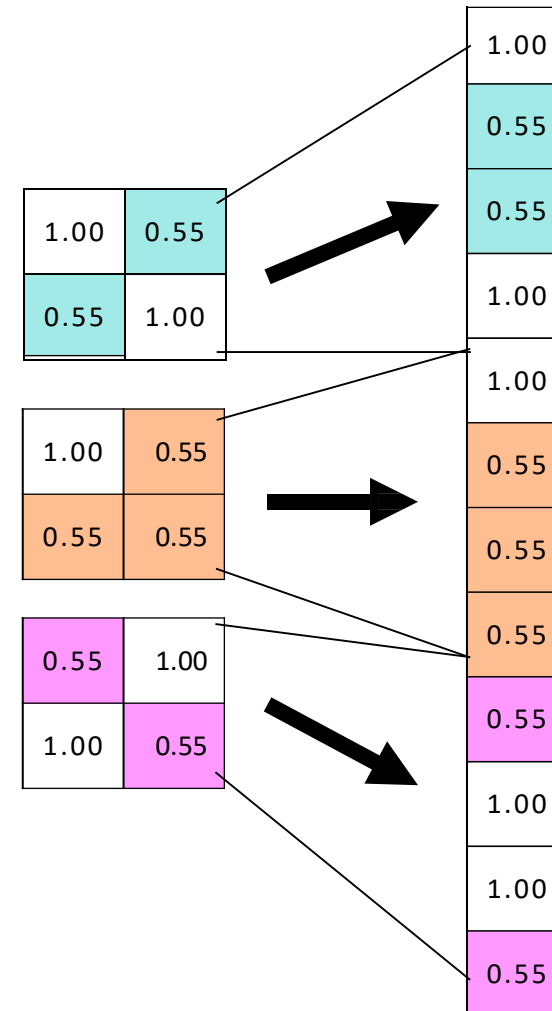
Deep stacking

Layers can be repeated several (or many) times.



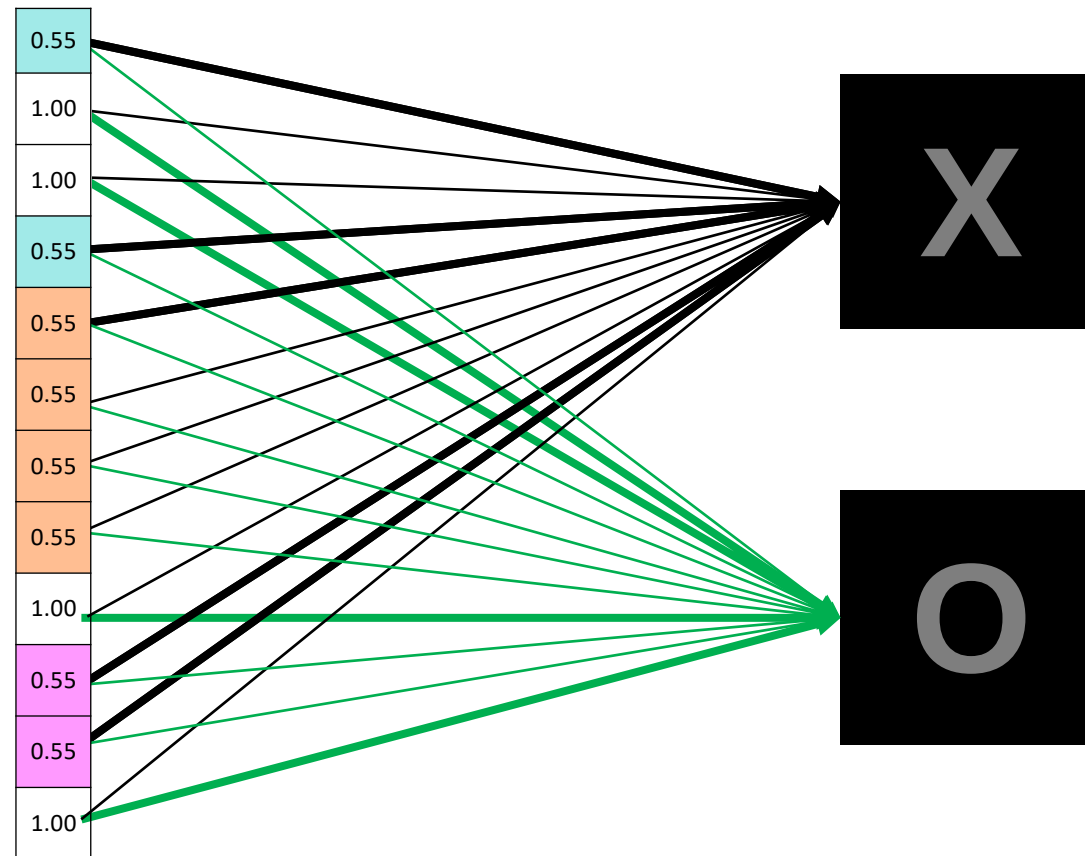
Fully connected layer

- Fully connected layers are the normal flat feed-forward neural network layers.
- These layers may have a non-linear activation function or a softmax activation in order to predict classes.
- To compute our output, we simply rearrange the output matrices as a 1-D array.

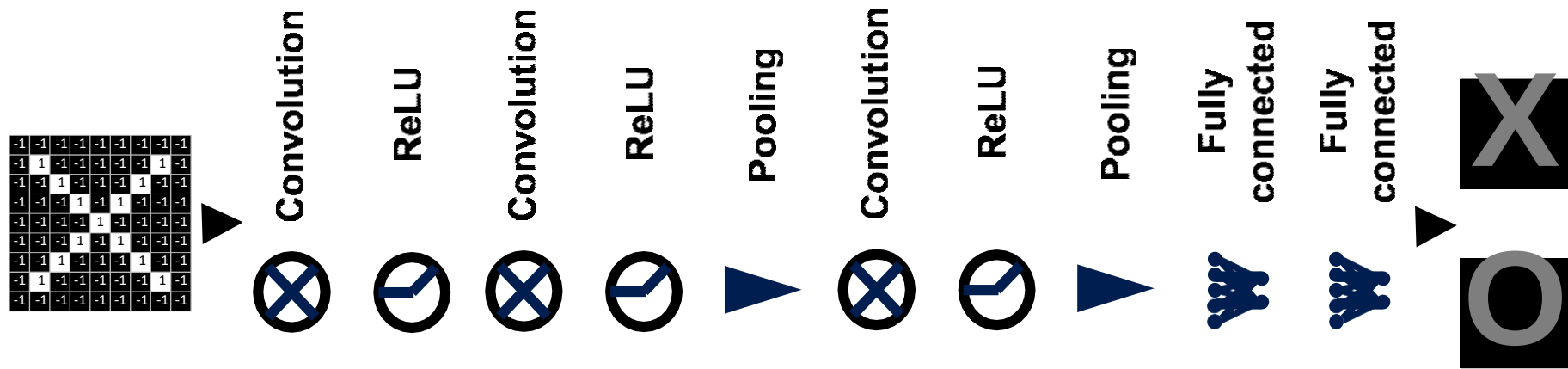


Fully connected layer

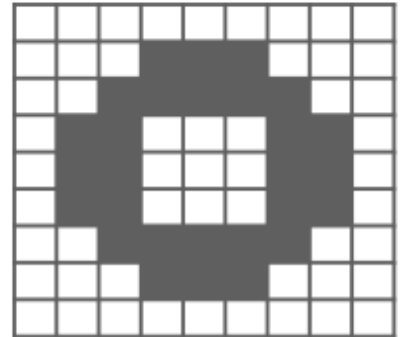
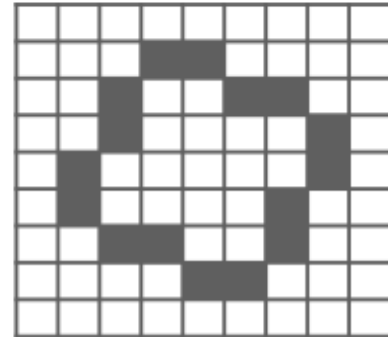
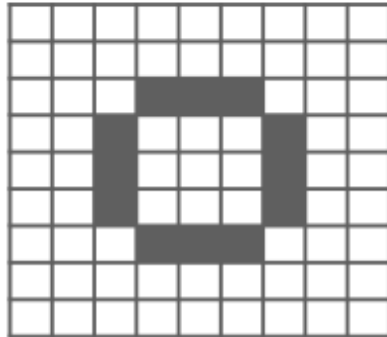
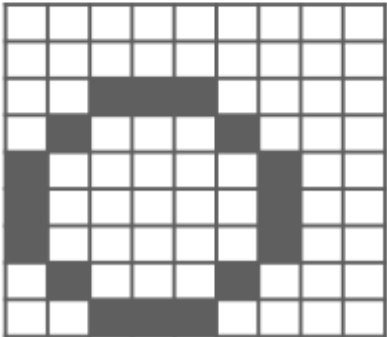
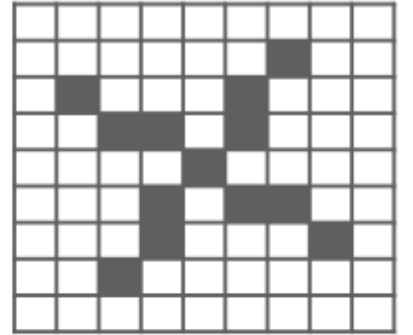
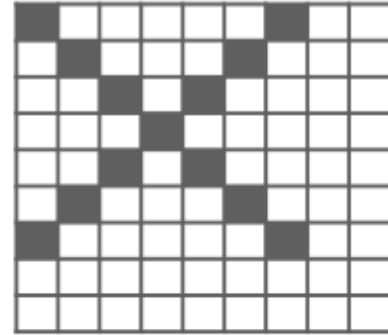
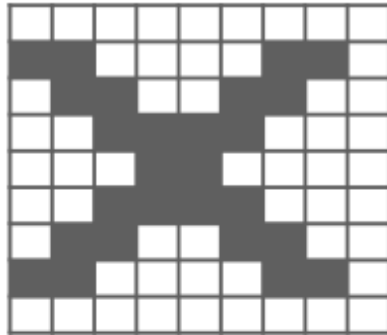
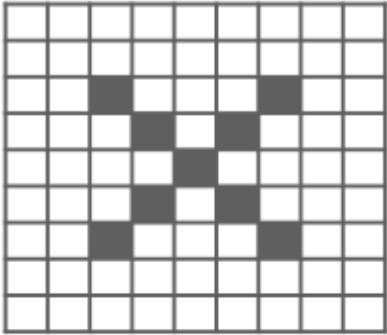
- A summation of product of inputs and weights at each output node determines the final prediction



Putting it all together



Example of CNN:



ReLu Function

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77



0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

Pooling Layer

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.33	0	0.11	0	0.11	0	0.33
0	0.55	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.33	0	1.00	0	0.33	0
0.11	0	0.55	0	0.55	0	0.11
0	0.55	0	0.33	0	0.55	0
0.33	0	0.11	0	0.11	0	0.33



0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

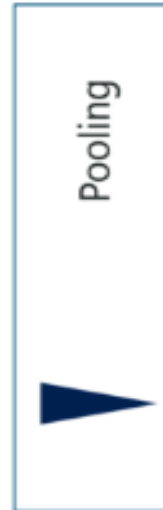
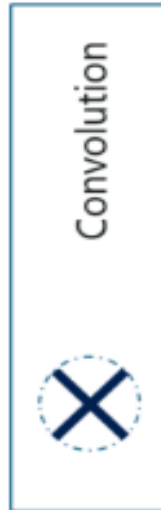
0.33	0	0.55	0.33	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.55	0	0.11	0	1.00	0	0.11
0.33	0.33	0	0.55	0	0.33	0.33
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.11	0
0.77	0	0.11	0.33	0.55	0	0.33



0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

Stacking Up The Layers

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

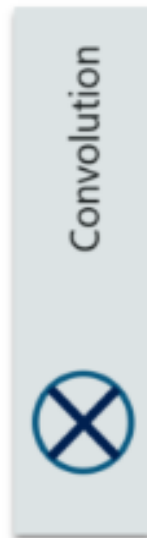
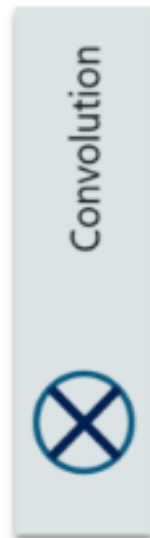


1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



1	0.55
0.55	1.00

1	0.55
0.55	0.55

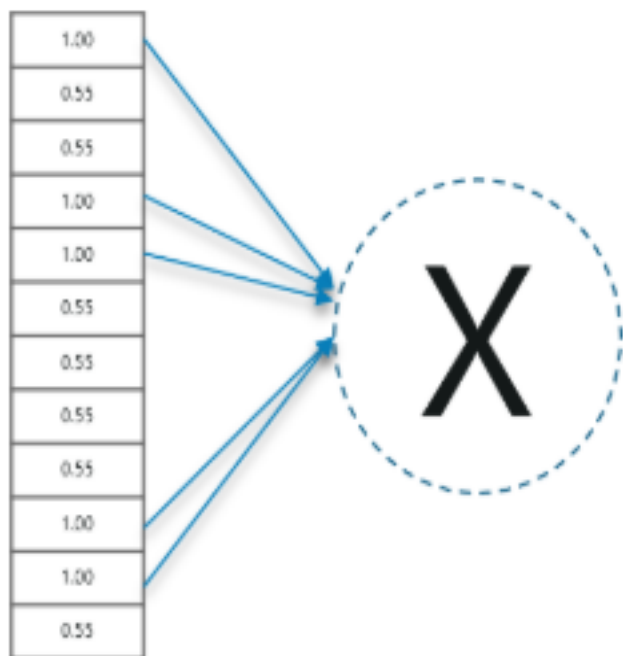
0.55	1.00
1.00	0.55

1	0.55
0.55	1.00

1	0.55
0.55	0.55

0.55	1.00
1.00	0.55

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
1.00
1.00
0.55



0.9
0.65
0.45
0.87
0.96
0.73
0.23
0.63
0.44
0.89
0.94
0.53

Sum



4.56



5

Sum



0.91

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
0.55
1.00
1.00
0.55

Input Image

Vector for 'X'

0.9
0.65
0.45
0.87
0.96
0.73
0.23
0.63
0.44
0.89
0.94
0.53

Input Image

Sum



2.07

/

4

0.51

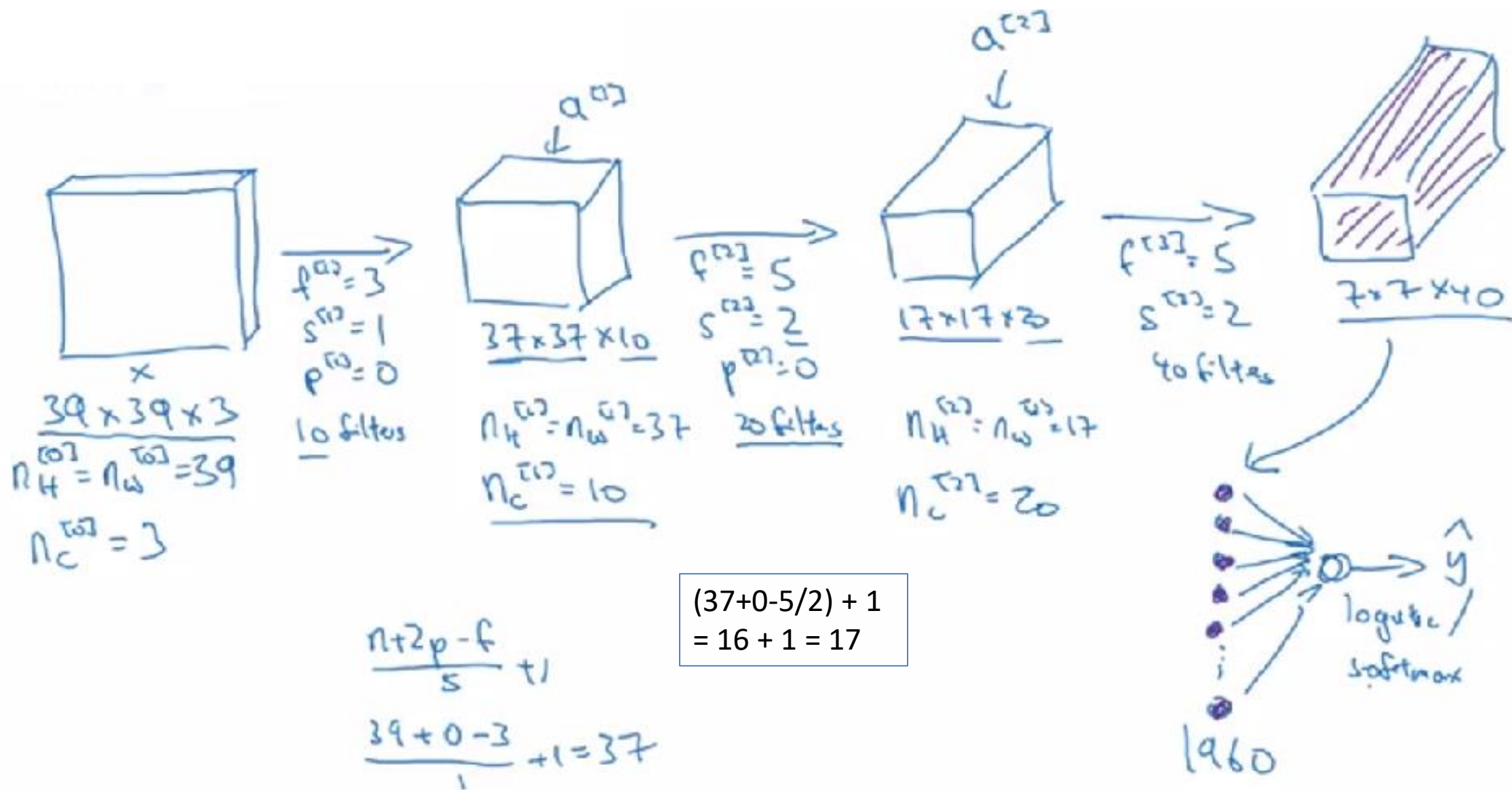
Sum



0.55
1.00
1.00
0.55
0.55
0.55
0.55
0.55
1.00
0.55
0.55
1.00

Vector for 'O'

Example of ConvNet



Different CNN Architectures

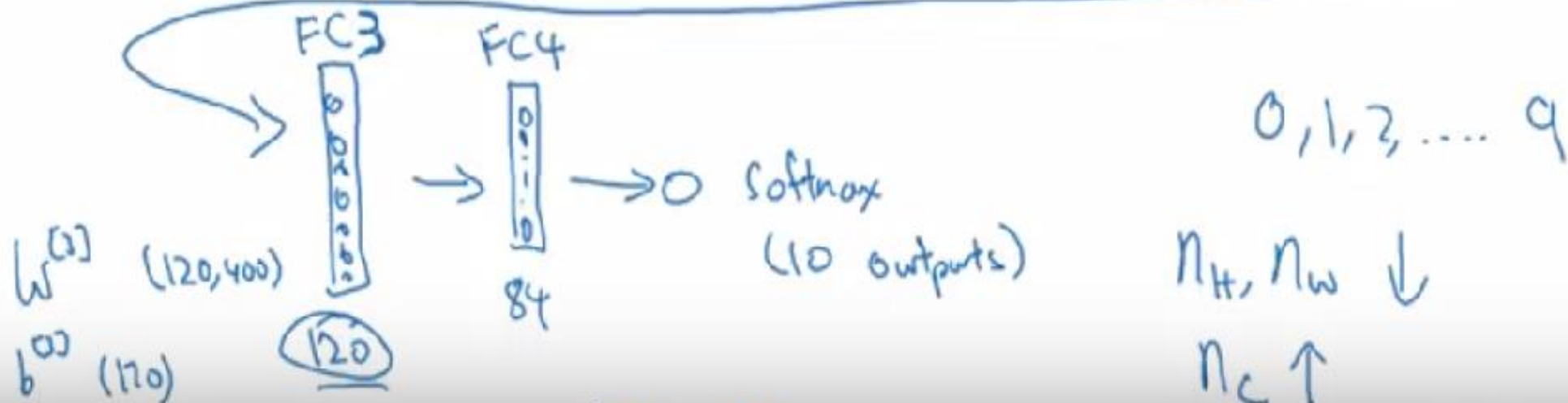
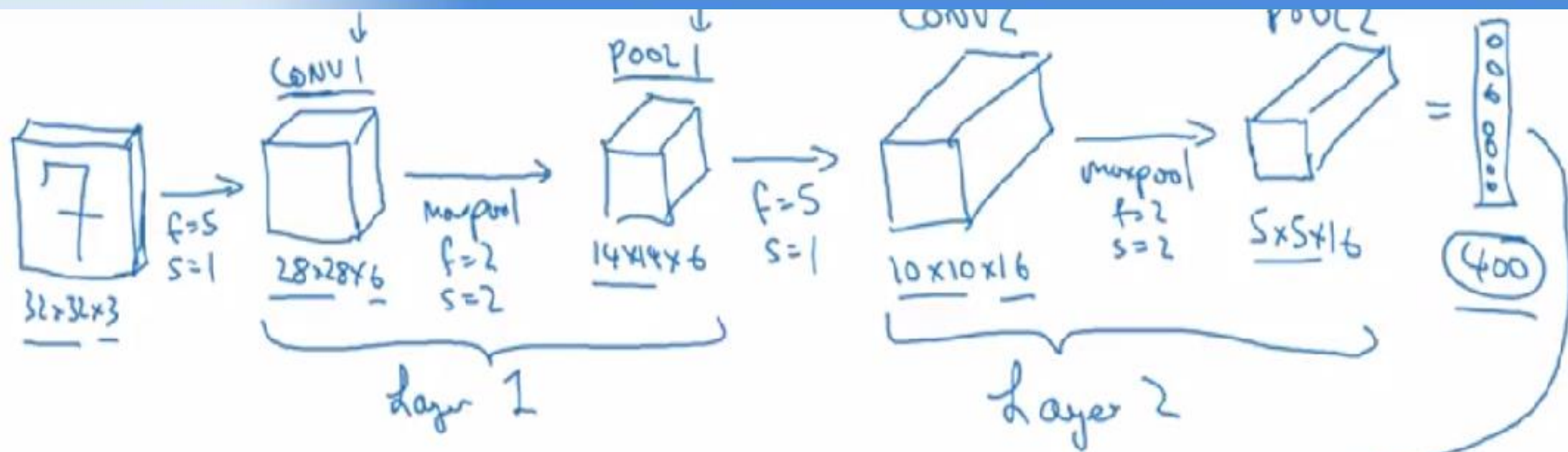
LeNet

Alexnet

VGG-19

ResNet

LeNet5



Sample Exercises:

CNN architecture for image classification: The input images are RGB images with dimensions 128x128 pixels.

Design a CNN architecture with the following components:

- Two convolutional layers with 3x3 filters, ReLU activation, and 32 filters each.
- Max pooling (2x2) after each convolutional layer.
- A fully connected layer with 128 neurons and ReLU activation.
- Output layer with 10 neurons and softmax activation for multiclass classification.
- Calculate the total number of parameters in the convolutional layers, the fully connected layer, and the entire network. Draw architecture and show your calculations step by step.

Hyperparameters (knobs)

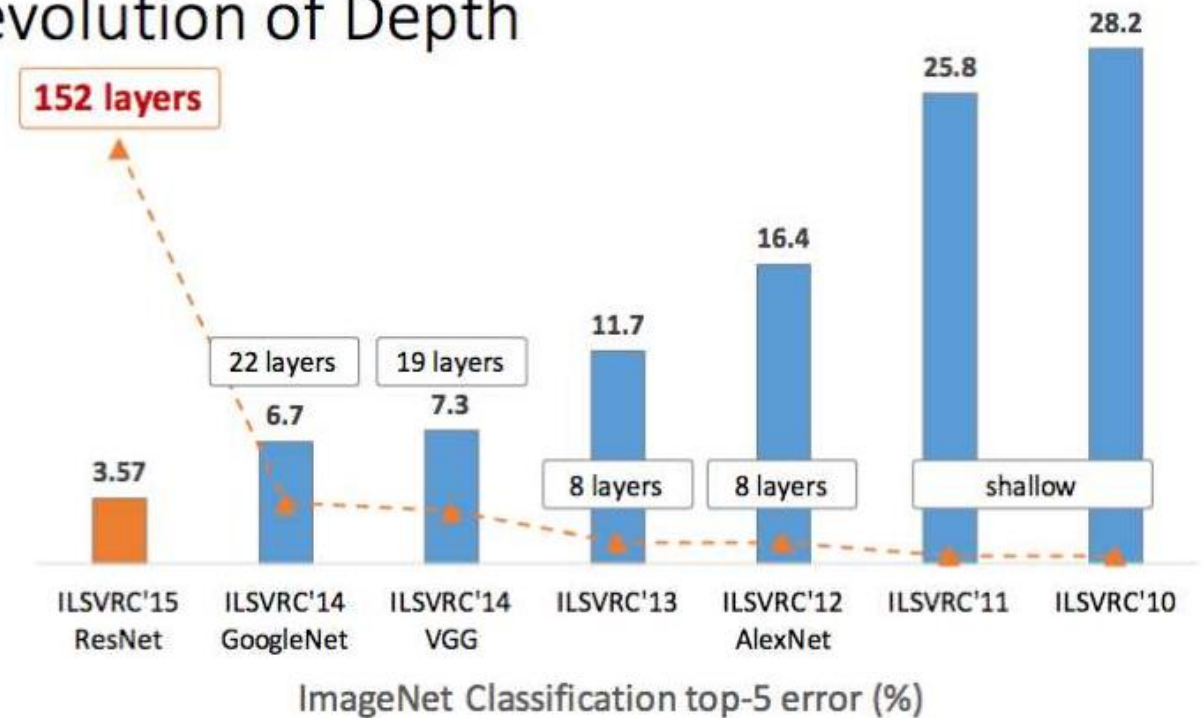
- Convolution
 - Filter Size
 - Number of Filters
 - Padding
 - Stride
- Pooling
 - Window Size
 - Stride
- Fully Connected
 - Number of neurons

Case Studies

- LeNet – 1998
- AlexNet – 2012
- ZFNet – 2013
- VGG – 2014
- GoogLeNet – 2014
- ResNet – 2015

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

Revolution of Depth



Deep vs Shallow Networks

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



56-layer model performs worse on both training and test error
-> The deeper model performs worse, but it's not caused by overfitting!

Deeper models are harder to optimize

The deeper model should be able to perform at least as well as the shallower model.

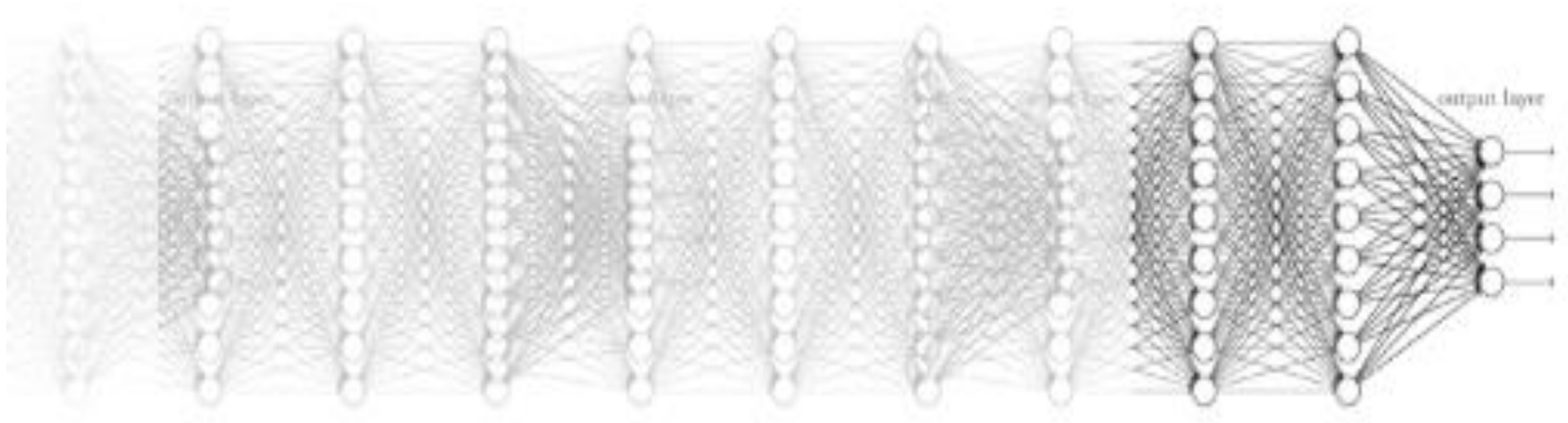
A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.

Challenges

- Deeper Neural Networks start to degrade in performance.
- Vanish/Exploding Gradient – May lead for extremely complex parameters initializations to make it work. Still might suffer from Vanish/Exploding even for the best parameters.
- Long training times – Due to too many training parameters.

Partial Solutions for Vanish/Exploding gradients

- Batch Normalization – To rescale the weights over some batch.
- Smart Initialization of weights – Like for example Xavier initialization.
- Train portions of the network individually.



Related Prior Work - Highway networks

- Adding features from previous time steps has been used in various tasks
- Most notable of these are Highway networks proposed by Srivastava et al.
- Highway networks feature residual connections Residual networks have the form

$$Y = f(x) + x$$

- Highway networks have the form

$$Y = f(x) \text{sigmoid}(Wx + b) + x(1 - \text{sigmoid}(Wx + b))$$

Highway networks – cont.

Highway networks enabled information flow from the past but due to the gating function

$$y = H(\mathbf{x}, \mathbf{W}_H) \cdot T(\mathbf{x}, \mathbf{W}_T) + \mathbf{x} \cdot (1 - T(\mathbf{x}, \mathbf{W}_T)).$$

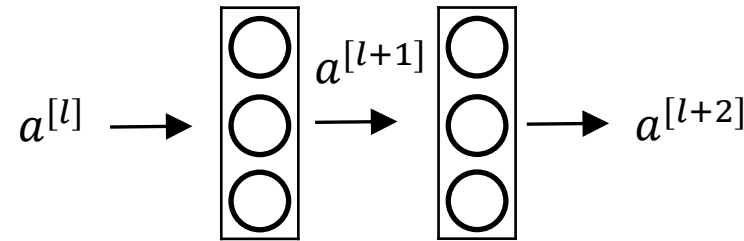
$$T(\mathbf{x}) = \sigma(\mathbf{W}_T^T \mathbf{x} + \mathbf{b}_T).$$

$$y = \begin{cases} \mathbf{x}, & \text{if } T(\mathbf{x}, \mathbf{W}_T) = 0, \\ H(\mathbf{x}, \mathbf{W}_H), & \text{if } T(\mathbf{x}, \mathbf{W}_T) = 1. \end{cases}$$

ResNet

- A specialized network introduced by Microsoft.
- Connects inputs of layers into farther part of that network to allow “shortcuts”.
- Simple idea – great improvements with both performance and train time.

Plain Network



$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

“linear”

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

“output”

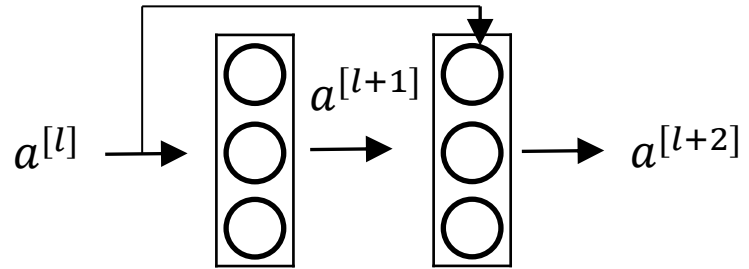
$$a^{[l+1]} = g(z^{[l+1]})$$

“relu”

$$a^{[l+2]} = g(z^{[l+2]})$$

“relu on output”

Residual Blocks



$$a^{[l+1]} = g(z^{[l+1]})$$

“relu”

$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

“linear”

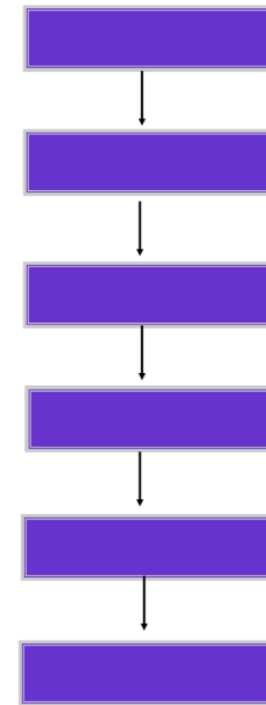
$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

“output”

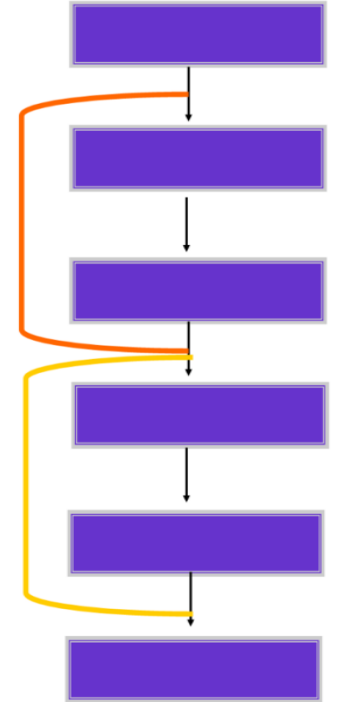
$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

“relu on output **plus input**”

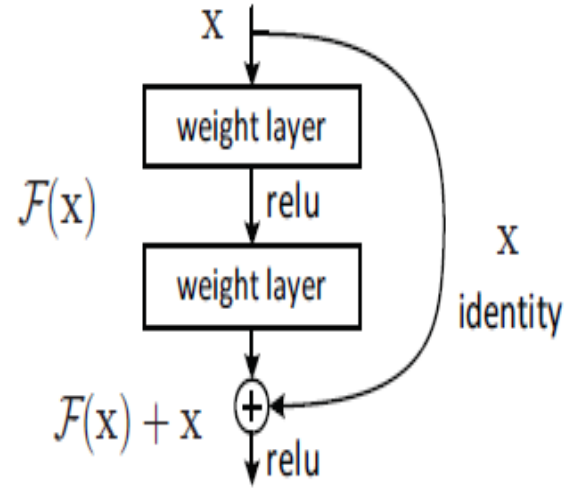
PLAIN CONNECTIONS



RESIDUAL CONNECTIONS



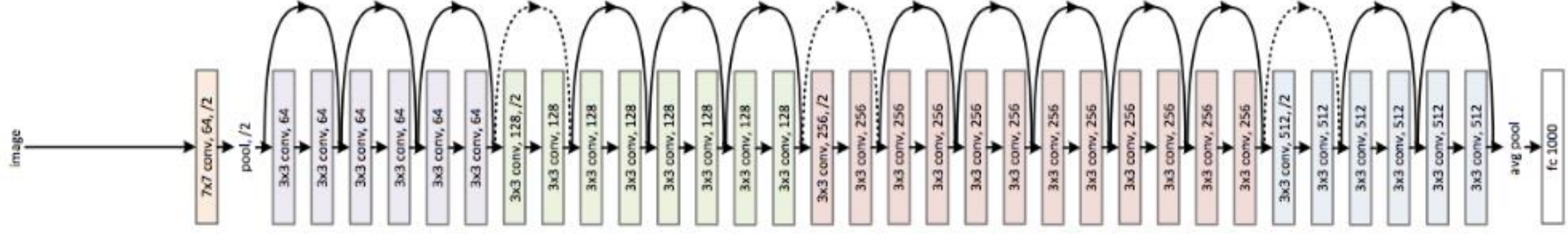
Skip Connections “shortcuts”



- Such connections are referred as skipped connections or shortcuts. In general similar models could skip over several layers.
- They refer to residual part of the network as a unit with input and output.
- Such residual part receives the input as an amplifier to its output – The dimensions usually are the same.
- Another option is to use a projection to the output space.
- Either way – no additional training parameters are used.

Residual Blocks (skip connections)

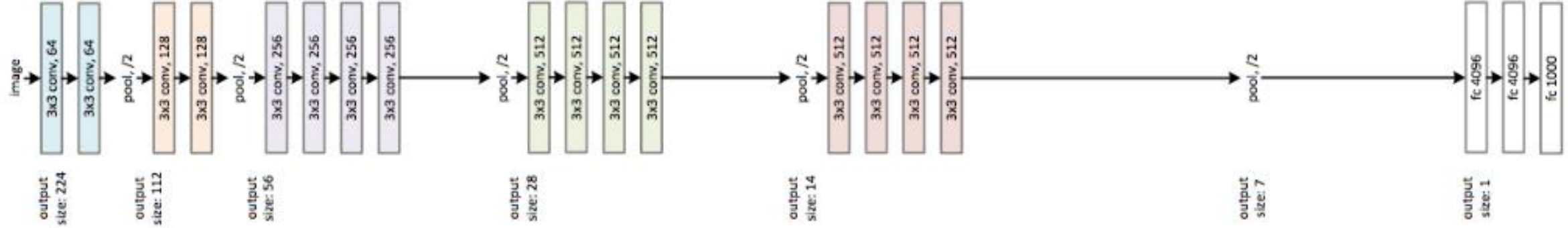
34-layer residual



34-layer plain



VGG-19



Deeper Bottleneck Architecture (Cont.)

- Addresses high training time of very deep networks.
- Keeps the time complexity same as the two layered convolution
- Allows us to increase the number of layers
- Allows the model to converge much faster.
- 152-layer ResNet has 11.3 billion FLOPS while VGG-16/19 nets has 15.3/19.6 billion FLOPS.

Why Do ResNets Work Well?

- Having a “regular” network that is very deep might actually hurt performance because of the vanishing and exploding gradients
- In most cases, ResNets will simply stop improving rather than decrease in performance
- $a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(w^{[l+1]}a^{[l+1]} + b^{[l]} + a^{[l]})$
- If the layer is not “useful”, L2 regularization will bring it’s parameters very close to zero, resulting in $a^{[l+2]} = g(a^{[l]}) = a^{[l]}$ (when using ReLU)

Why Do ResNets Work Well? (Cont)

- In theory ResNet is still identical to plain networks, but in practice due to the above the convergence is much faster.
- No additional training parameters introduced.
- No additional complexity introduced.

Training ResNet in practice

- Batch Normalization after every CONV layer.
- Xavier/2 initialization from He et al.
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus.
- Mini-batch size 256.
- Weight decay of $1e-5$.
- No dropout used.

Loss Function

- For measuring the loss of the model a combination of cross-entropy and softmax were used.
- The output of the cross-entropy was normalized using softmax function.

$$H(p, q) = - \sum_x p(x) \log q(x).$$

$$\sigma : \mathbb{R}^K \rightarrow \left\{ z \in \mathbb{R}^K \mid z_i > 0, \sum_{i=1}^K z_i = 1 \right\}$$

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

Reduce Learning Time with Random Layer Drops

- Dropping layers during training, and using the full network in testing.
- Residual block are used as network's building block.
- During training, input flows through both the shortcut and the weights.
- Training: Each layer has a “survival probability” and is randomly dropped.
- Testing: all blocks are kept active.
- Re-calibrated according to its survival probability during training.

Thank you!!